14 haftalık yol haritasında 6. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 06

Böl ve Fethet

Birleştirmeli Sıralama, Hızlı Sıralama ve Yineleme Bağıntıları

MODÜL II · ARAMA VE SIRALAMA · H4–H7

> **HAFTANIN BÜYÜK SORUSU**
>
> *Bir milyar kaydı sıralamak neden bir milyar kere bir milyar işlem gerektirmez ve "yarıya bölmek" bu farkı nasıl yaratır?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 4
1.1 Öğrenme çıktıları 4
2. Geçen Haftadan ve AP-1'den Bağlantı 5
3. Kavramsal Anlatım: Böl ve Fethet ve Yineleme Bağıntıları 5
3.1 Böl ve fethet paradigması 5
3.2 Yineleme bağıntıları ve yerine koyma yöntemi 6
3.3 Özyineleme ağacı yöntemi 6
3.4 Ana teorem (basitleştirilmiş) 7
3.5 Karar ağacı ve Ω(n log n) alt sınırı 8
3.6 Kaputun Altında: tampon, yerinde bölümleme, yığın ve kütüphaneler 9
4. Algoritmalar 10
4.1 Birleştirme ve birleştirmeli sıralama 10
4.2 Hızlı sıralama ve bölümleme 14
4.3 Ters çevrim sayısı 21
4.4 Quickselect: k'ıncı en küçük ve ortanca 23
4.5 En büyük alt dizi toplamı (böl ve fethet) 26
5. Çözümlü Örnekler 28
5.1 Aşağıdan yukarıya birleştirmeli sıralama (Örnek 10) 29
5.2 Melez sıralama: küçük parçalarda araya ekleme (Örnek 11) 30
5.3 Büyük deney programı (Örnek 12) 31
6. Haftanın Deneyi: n log n ile n² Karşı Karşıya 31
6.1 Karesel çöküş: sıralı girdi ve ilk eleman pivotu 33
6.2 Kestirim yöntemi ve sınanması 33
6.3 Melez sıralamada eşik 33
7. Hata Avcısı 34
8. Dil Penceresi: C · C++ · Python 36
9. Mühendis Gözüyle 37
10. Mini-OBS 2 v2.5: Büyük Veride Böl ve Fethet 37
11. Alıştırmalar ve Atölyeler 40
11.1 İzleme ve çıktı tahmini (A) 40
11.2 Karmaşıklık (B) 40
11.3 Değişmez yazma (C) 41
11.4 Hata bulma (D) 41
11.5 Parsons (E) 42

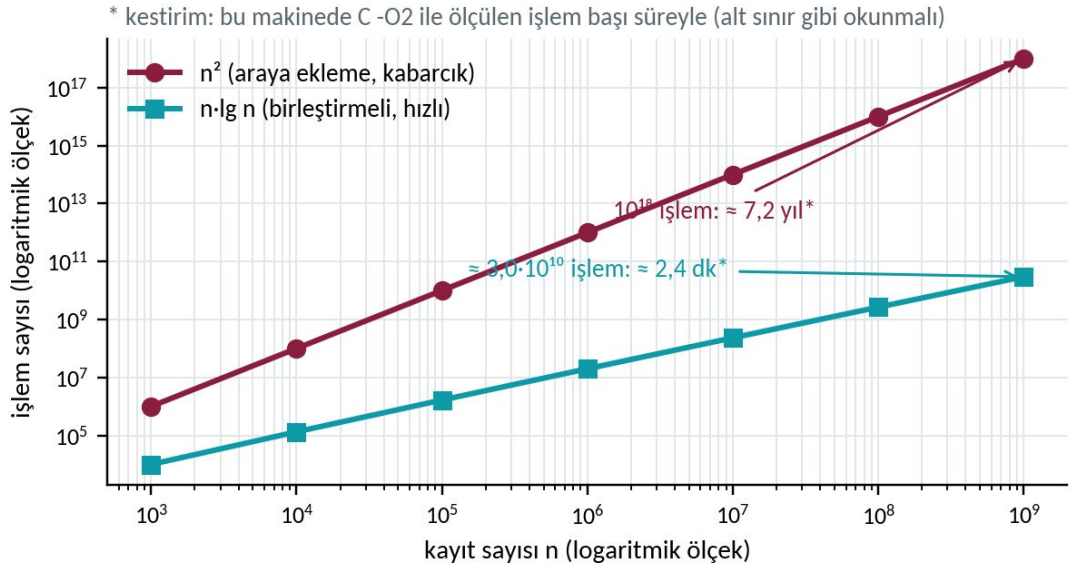
11.6 Kod yazma (F)..... 43
11.7 Diller arası okuma (G)..... 43
11.8 Atölye 1 · Kâğıt birleştirme (gruplar, ≈ 20 dk)..... 44
11.9 Atölye 2 · Pivot avı: hızlı sıralamayı karesel davranışa zorlayın (gruplar, ≈ 25 dk)..... 44
11.10 Atölye 3 · Büyük sıralama yarışı: 10^7 tamsayıyı en hızlı kim sıralar? (gruplar, ≈ 30 dk)..... 45
12. Kendini Değerlendirme..... 45
13. İleri Bakış ve Merak Kutusu..... 46
14. Özet, Kavram Haritası ve Terim Sözlüğü..... 46
15. Kaynaklar..... 48
Ek A. Dil Karşılıkları..... 49
A.1 Örnek 1 · İki sıralı dizinin birleştirilmesi: birleştirme değişmezi ve eşitlikte soldan alarak kararlılık..... 49
A.2 Örnek 2 · Birleştirmeli sıralama (yukarıdan aşağıya, tek yardımcı tampon): karşılaştırma, kopya, çağrı ve derinlik..... 50
A.3 Örnek 3 · Özyineleme ağacı izleme: düzey başına iş sayacı ve ana teoremin üç durumu..... 51
A.4 Örnek 4 · Lomuto bölümleme (pivot = a[hi]) ve hızlı sıralama: değişmez, izleme ve 2(n+1)H(n) - 4n beklentisi..... 52
A.5 Örnek 5 · Hoare bölümleme ve Lomuto ile karşılaştırma/takas sayısı; dönüş indisi pivotun yeri değildir..... 53
A.6 Örnek 6 · Pivot deneyi: ilk eleman / rastgele / üçün ortancası × sıralı, ters, rastgele girdi; ağaç yüksekliği ve yığın derinliği..... 54
A.7 Örnek 7 · Üç yollu bölümleme (Hollanda bayrağı, Dijkstra) ve çok tekrar eden anahtarlarda iki yollu ile karşılaştırma..... 54
A.8 Örnek 8 · Ters çevrim sayısı: birleştirmeli sıralamayla $\Theta(n \log n)$; H5'in $\Theta(n^2)$ çift taraması kâhin 55
A.9 Örnek 9 · Quickselect: ortanca not (üç yollu bölümleme, rastgele pivot, yinelemeli); sıralama kâhine karşı..... 57
A.10 Örnek 10 · Aşağıdan yukarıya (yinelemeli) birleştirmeli sıralama: geçişler, eşsiz son blok ve yukarıdan aşağıya ile karşılaştırma..... 58
A.11 Örnek 11 · Melez sıralama: hızlı sıralama + küçük parçalarda araya ekleme; eşik deneyi (4… 64; sayaç ve süre)..... 59
A.12 Örnek 12 · Büyük deney programı: birleştirmeli / hızlı (3 pivot) / kütüphane / araya ekleme; varsayılan küçük, ölçüm kipi CSV..... 60
A.13 Örnek 13 · Böl ve fethet ile en büyük alt dizi toplamı; H2 Kadane ile aynı sonuç, toplama sayısı karşılaştırması..... 61

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir milyar kaydı sıralamak neden bir milyar kere bir milyar işlem gerektirmez ve "yarıya bölmek" bu farkı nasıl yaratır?

Bir milyar ($n = 10^9$) kaydı geçen haftanın araya ekleme sıralamasıyla sıralamak kabaca $n^2 = 10^{18}$ temel işlem ister; bu haftanın birleştirmeli ve hızlı sıralaması $n \cdot \lg n \approx 2{,}99 \cdot 10^{10}$ işlemle yetinir: oran $n / \lg n \approx 3{,}35 \cdot 10^7$ (Kavram 1). Bu makinede C ile (-O2) ölçülen işlem başı süreyle 10^9 kayıt için hızlı sıralama ≈ 2,4 dakika, araya ekleme ≈ 7,2 yıl sürerdi (**kestirim**; bellek ve önbellek etkileri eklenmedi, alt sınır gibi okunmalı). Ölçebildiğimiz en büyük $n = 10^7$'de birleştirmeli sıralama 1,51 s, üçün ortancası pivotlu hızlı sıralama 1,11 s sürdü; araya eklemenin kestirimi ≈ 6,3 sa (Bölüm 6).

Şekil 6.1
n^2 ile $n \cdot \lg n$: 10^3 … 10^9 kayıt için işlem sayısı (log–log); 10^9 için süreler bu makinede ölçülen işlem başı süreyle kestirildi



Farkı yaratan fikir **böl ve fethet**tir (divide and conquer): problemi aynı türden küçük parçalara **böl**, parçaları aynı yöntemle **fethet**, çözümleri **birleştir**. Yarıya bölmek derinliği lg n ile sınırlar; her düzeydeki iş doğrusal kalırsa toplam $n \cdot \lg n$ olur. Bu fikri iki sıralamada, bir seçim algoritmasında (quickselect) ve iki problemde (ters çevrim sayısı, en büyük alt dizi) uygulayacak, maliyetleri **yineleme bağıntıları** ve **ana teoremle** çözecek ve karşılaştırmaya dayalı sıralamanın neden $n \cdot \lg n$'den hızlı olamayacağını göreceğiz.

1.1 Öğrenme çıktıları

- **ÖÇ 6.1** Böl ve fethet stratejisini üç adımda (böl, fethet, birleştir) ifade eder ve problemlere uygular. (Bölüm 3.1, 4)
- **ÖÇ 6.2** İki sıralı diziyi doğrusal zamanda birleştirir; birleştirmeli sıralamayı gerçekler ve O(n log n) karmaşıklığını özyineleme ağacıyla gösterir. (Bölüm 4.1, 5.1, Örnek 1–3, 10)
- **ÖÇ 6.3** Lomuto ve Hoare bölümleme yöntemleriyle hızlı sıralamayı gerçekler; pivot seçiminin en iyi, en kötü ve ortalama duruma etkisini açıklar. (Bölüm 4.2, Örnek 4–7, 11)
- **ÖÇ 6.4** Böl ve fethet yineleme bağıntılarını özyineleme ağacı ve basitleştirilmiş ana teoremle çözer. (Bölüm 3.2–3.4, Örnek 3)
- **ÖÇ 6.5** Karşılaştırmaya dayalı sıralamanın Ω(n log n) alt sınırını karar ağacıyla açıklar; quickselect ile k'ıncı en küçük elemanı ortalama doğrusal zamanda bulur. (Bölüm 3.5, 4.4, Örnek 9)

GERÇEK DÜNYADAN • **Moskova 1959: sözlükteki sözcükleri sıralamak**

Hızlı sıralama (quicksort), C. A. R. (Tony) Hoare'un 1959'da Moskova Devlet Üniversitesi'nde öğrenciyken geliştirdiği algoritmadır (Spicer, 2026). Hoare'un anlatımına göre o sırada National Physical Laboratory'nin Rusçadan İngilizceye makine çevirisi projesi için çalışıyordu: sözlük manyetik bantta alfabetik sırada durduğundan bir cümlenin sözcükleri önce sıralanırsa banttan **tek geçişte** aranabilirdi. İlk aklına gelen yöntem (bugünkü adıyla kabarcık sıralaması) yavaştı; hızlı sıralama ikinci fikriydi (Shustek, 2009).

İngiltere'de Elliott Brothers'ta çalışırken patronuyla, Shell'in yönteminden hızlı bir sıralama bildiğine dair altı penilik bir iddiaya girip kazandı, ama yöntemi açıklamakta zorlandı; 1961 Paskalyası civarında Brighton'daki bir ALGOL 60 kursunda özyinelemeli yordamları öğrenince algoritmayı kolayca yazabildi (Hoare, 1981). Algoritma 1961'de yayımlandı, çözümlemesi 1962'de (Hoare, 1961a, 1962); aynı sayıdaki FIND yordamı bu haftanın quickselect'idir (Hoare, 1961b). Hoare, 5 Mart 2026'da 92 yaşında hayatını kaybetti (Spicer, 2026).

Birleştirmeli sıralama 1945'te John von Neumann'a atfedilir (Knuth, 1998); Knuth (1970), von Neumann'ın EDVAC için yazdığı sıralama ve birleştirme programını incelemiştir. McIlroy (1999) ise girdiyi karşılaştırmalara yanıt verirken kurgulayan bir "katil rakip" (killer adversary) ile pek çok qsort gerçekleştiriminin karesel davranışa zorlanabileceğini gösterdi (Bölüm 9, Atölye 2).

2. Geçen Haftadan ve AP-1'den Bağlantı

H5'teki seçmeli, kabarcık ve araya ekleme sıralamaları en kötü durumda $\Theta(n^2)$ karşılaştırma yapar; araya eklemenin kaydırma sayısı ters çevrim (inversion) sayısına eşittir ve algoritma **kararlıdır** (stable): eşit anahtarlı kayıtların göreli sırasını korur. Bu hafta işi yarıya bölerek $\Theta(n \log n)$'e iniyor ve farkı bir **yineleme bağıntısıyla** (recurrence) gösteriyoruz. Geriye dönük üç bağlantı var: **H4'teki ikili arama** bir böl ve fethet algoritmasıdır ($T(n) = T(n/2) + 1$); **H3'teki özyineleme ağacı ve yerine koyma yöntemi** bu haftanın analiz araçlarıdır (orada $T(n) = 2T(n-1) + 1$ gibi "bir eksilten" bağıntılar üstel ağaçlar vermişti, burada "yarıya bölen" bağıntılar lg n yükseklikli ağaçlar verecek); **AP-1 H9'daki Hanoi Kuleleri** de "büyük problemi aynı türden küçük problemlere indir" fikrinin ilk örneğiydi.

TAHMİN ET • **Geri çağırma: üç soru (yanıtlar aşağıda)**

- **S1 (H5).** Araya ekleme [6, 3, 8, 1, 7, 2, 5, 4] dizisini sıralarken kaç kaydırma yapar?
- **S2 (H4).** n = 1024 elemanlı sıralı dizide ikili arama en kötü durumda kaç kez ortadaki elemana bakar? Bağıntıyı yazın.
- **S3 (H3).** $T(n) = 2T(n-1) + 1$, $T(0) = 0$ bağıntısının çözümü nedir? $T(10)$ kaçtır?

Yanıtlar. S1: kaydırma sayısı ters çevrim sayısına eşittir: 16 (Örnek 8 bunu n log n'de sayacak). S2: $T(n) = T(n/2) + 1 \rightarrow \lg n + 1$; n = 1024 için 11 (Kavram 2). S3: $2^n - 1$; $T(10) = 1023$.

3. Kavramsal Anlatım: Böl ve Fethet ve Yineleme Bağıntıları

3.1 Böl ve fethet paradigması

Böl: problemi aynı türden daha küçük alt problemlere ayır. **Fethet:** alt problemleri özyinelemeyle çöz; yeterince küçük olanı (temel durum) doğrudan çöz. **Birleştir:** alt çözümlerden asıl çözümü kur (Cormen vd., 2022). Maliyet, işin bu üç adıma nasıl dağıldığına bağlıdır. Birleştirmeli sıralama bölmeyi bedavaya (ortadan ikiye) yapar ve asıl işi **birleştirmede** yapar; hızlı sıralama asıl işi **bölmede** (bölümleme) yapar ve birleştirmeye gerek bırakmaz. İkili arama (bir yarı, birleştirme yok), quickselect (yalnızca k'yı içeren parça) ve en büyük alt dizi (iki yarı + ortayı keseni bulma) aynı kalıbın başka örnekleridir.

3.2 Yineleme bağıntıları ve yerine koyma yöntemi

Özyinelemeli bir algoritmanın n boyutlu girdideki maliyeti T(n), küçük girdilerdeki maliyetlerle yazılır. Birleştirmeli sıralama iki yarıyı sıralar (2·T(n/2)) ve birleştirir (n işlem): **T(n) = 2T(n/2) + n**, T(1) = 1. **Yerine koyma** (substitution) yöntemi H3'ten bilinir: bağıntıyı kendi içine yerleştir, k adımın genel biçimini yaz, k'yı temel duruma ulaşacak biçimde seç. $n = 2^k$ için:

- T(n) = 2T(n/2) + n = 4T(n/4) + 2n = … = 2^k·T($n/2^k$) + k·n; k = lg n'de **T(n) = n + n·lg n = Θ(n log n)**.
- T(n) = T(n − 1) + n (pivot hep uçta) = T(n − 2) + (n − 1) + n = … = **n(n + 1)/2 = Θ(n^2)**.
- T(n) = T(n/2) + 1 (ikili arama) = T(n/4) + 2 = … = **lg n + 1 = Θ(log n)**.
- T(n) = 2T(n/2) + 1 (iki yarı, sabit birleştirme; ör. en büyük eleman) = … = n + (n − 1) = **2n − 1 = Θ(n)**.

Son iki satır böl ve fethetin her zaman n log n vermediğini gösterir: sonuç alt problem sayısına (a), küçülme çarpanına (b) ve bölme + birleştirme işine bağlıdır. Kavram 2 dört bağıntıyı doğrudan özyinelemeyle hesaplayıp kapalı biçimlerle karşılaştırır:

ÇIKTI · KAVRAM 2

```
$ ./H06_K02_bagintilar   (seçili satırlar)
n = 2^k   2T(n/2)+n  n·k+n   T(n-1)+n  n(n+1)/2  T(n/2)+1  k+1  2T(n/2)+1  2n-1
      1          1      1          1         1         1    1          1     1
      8         32     32         36        36         4    4         15    15
     64        448    448       2080      2080         7    7        127   127
   1024      11264  11264     524800    524800        11   11       2047  2047
   4096      53248  53248    8390656   8390656        13   13       8191  8191
kapalı biçimler bağıntılarla aynı
```

3.3 Özyineleme ağacı yöntemi

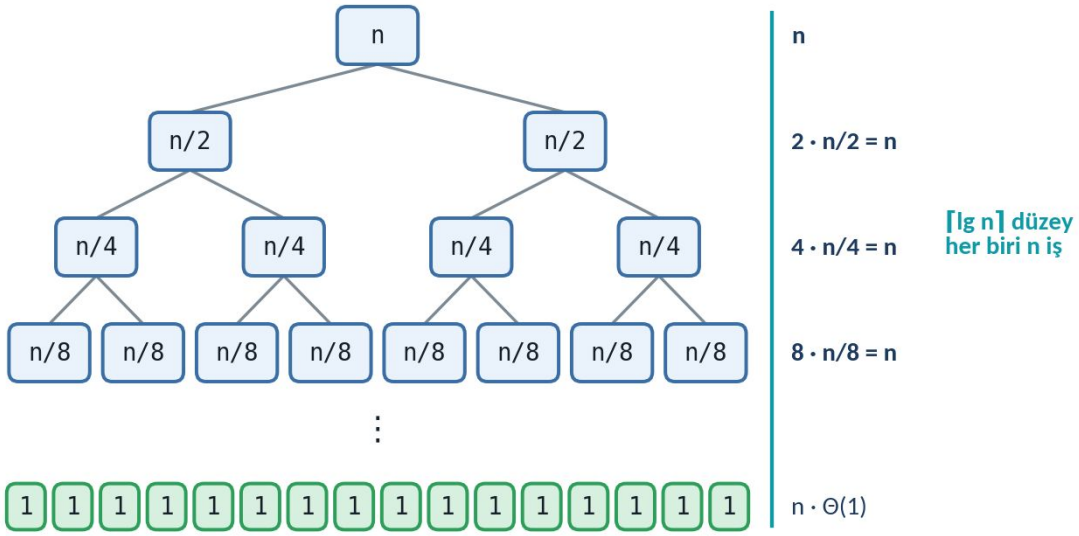
Özyineleme ağacı (recursion tree) aynı hesabı görselleştirir: her düğüm bir çağrıdır ve kendi bölme + birleştirme işini taşır; toplam maliyet düğüm işlerinin toplamıdır. Düğümleri **düzey düzey** toplamak hesabı kolaylaştırır: T(n) = 2T(n/2) + n ağacında d. düzeyde 2^d düğüm vardır, her biri $n/2^d$ iş yapar; düzeyin toplam işi yine n'dir ve ⌈lg n⌉ düzey vardır (Şekil 6.2, aşağıda).

Örnek 3 bu ağacı birleştirmeli sıralamanın gerçek yürütmesinden ölçer: `bs_izle` her çağrıda düzeyini taşır ve birleştirme bittiğinde düğümün işini (birleştirdiği eleman sayısı) kendi düzeyine ekler: `duzey_is[d] += boyut;` (satır 93).

ÇIKTI · ÖRNEK 3

```
$ ./H06_03_ozyineleme_agaci   (n = 1024 bölümü, seçili satırlar)
n = 1024
düzey  düğüm   boyut      iş  karşılaştırma
    1      1    1024    1024          1022
    2      2     512    1024          1022
    3      4     256    1024          1018
    9    256       4    1024           679
   10    512       2    1024           512
   11   1024       1       0             0
toplam iş = 10240 · n·⌈lg n⌉ = 1024·10 = 10240 · karşılaştırma = 8944
```

Şekil 6.2
T(n) = 2T(n/2) + n özyineleme ağacı: her düzeyde toplam iş n, ⌈lg n⌉ düzey; yapraklarda n tane Θ(1) iş



T(n) = 2T(n/2) + n → toplam ≈ n·lg n (Örnek 3: n = 1024 için iş yapan her düzeyde 1024, toplam 10240)

n = 1024'te iş yapan her düzeyde iş tam 1024'tür; toplam 10 · 1024 = n·⌈lg n⌉. İkinin kuvveti olmayan n = 1000'de son düzey eksik kalır: toplam 9 976 ≤ 10 000. Bir birleştirme, birleştirdiği eleman sayısından en çok 1 eksik karşılaştırma yapar (bir dizi bitince kalan karşılaştırmasız kopyalanır); bu yüzden karşılaştırmalar işten biraz azdır.

3.4 Ana teorem (basitleştirilmiş)

Böl ve fethet bağıntılarının çoğu **T(n) = a·T(n/b) + Θ(n^d)** kalıbındadır: a ≥ 1 alt problem sayısı, b > 1 küçülme çarpanı, Θ(n^d) bölme + birleştirme işi (d ≥ 0). Ağacın k. düzeyinde a^k düğüm vardır, her biri (n/b^k)^d iş yapar: düzey işi n^d·(a/b^d)^k, yani düzey işleri **oranı a/b^d olan bir geometrik dizidir** ve sonucu üç durumdan biri belirler (Cormen vd., 2022; Sedgewick & Wayne, 2011):

- **a < b^d (kök baskın):** düzey işleri azalır; toplamı kökün işi belirler: **T(n) = Θ(n^d)**.
- **a = b^d (düzeyler eşit):** her düzeyde n^d iş, log_b n düzey: **T(n) = Θ(n^d log n)**.
- **a > b^d (yapraklar baskın):** düzey işleri artar; toplamı yaprak sayısı a^(log_b n) = n^(log_b a) belirler: **T(n) = Θ(n^(log_b a))**.

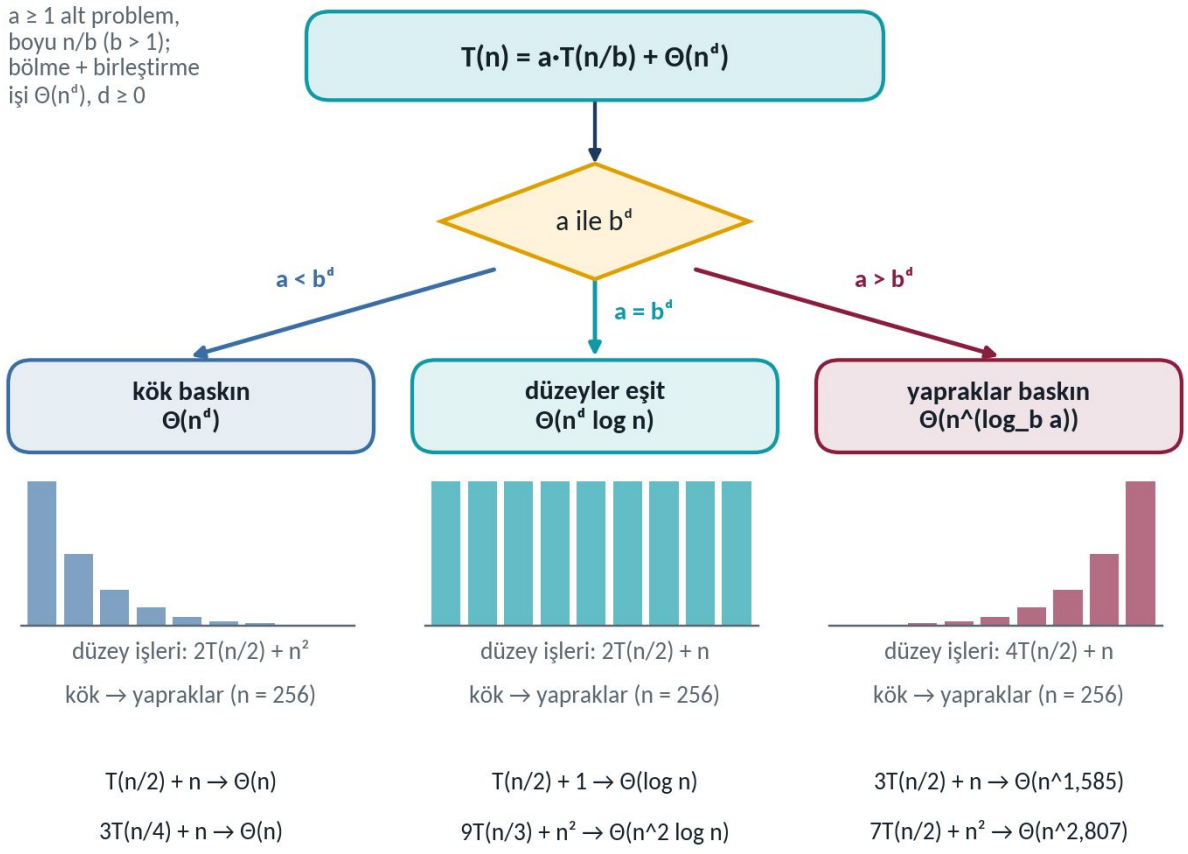
Örnek 3'ün `yineleme` fonksiyonu bağıntıyı gerçekten özyinelemeyle çalıştırır (her çağrı kendi düzeyine n^d ekler ve a kez n/b boyutuyla kendini çağırır); karar a ile b^d'nin **tamsayı** karşılaştırmasıyla verilir. Tablo 6.1'deki her satır n = b^k için bu yolla doğrulandı.

Tablo 6.1
Ana teoremin uygulanışı (Örnek 3 ve izleme verisi)

Bağıntı	a, b, d	b^d	Durum	Çözüm	Nerede?
2T(n/2) + n	2, 2, 1	2	düzeyler eşit	Θ(n log n)	birleştirmeli sıralama, en büyük alt dizi
T(n/2) + 1	1, 2, 0	1	düzeyler eşit	Θ(log n)	ikili arama (H4)
3T(n/2) + n	3, 2, 1	2	yapraklar baskın	Θ(n^1,585)	Karatsuba çarpımı (Alıştırma 6.14)
7T(n/2) + n²	7, 2, 2	4	yapraklar baskın	Θ(n^2,807)	Strassen matris çarpımı

Bağıntı	a, b, d	b^d	Durum	Çözüm	Nerede?
$8T(n/2) + n^2$	8, 2, 2	4	yapraklar baskın	Θ(n^3)	naif bölünmüş matris çarpımı
$T(n/2) + n$	1, 2, 1	2	kök baskın	Θ(n)	quickselect, iyi pivot sezgisi

Şekil 6.3
Basitleştirilmiş ana teoremin karar şeması: a ile b^d karşılaştırılır; çubuklar Örnek 3'ün $n = 256$ için gerçek özyinelemeyle ölçtüğü düzey işleri (kökten yapraklara): $2T(n/2) + n^2$, $2T(n/2) + n$ ve $4T(n/2) + n$

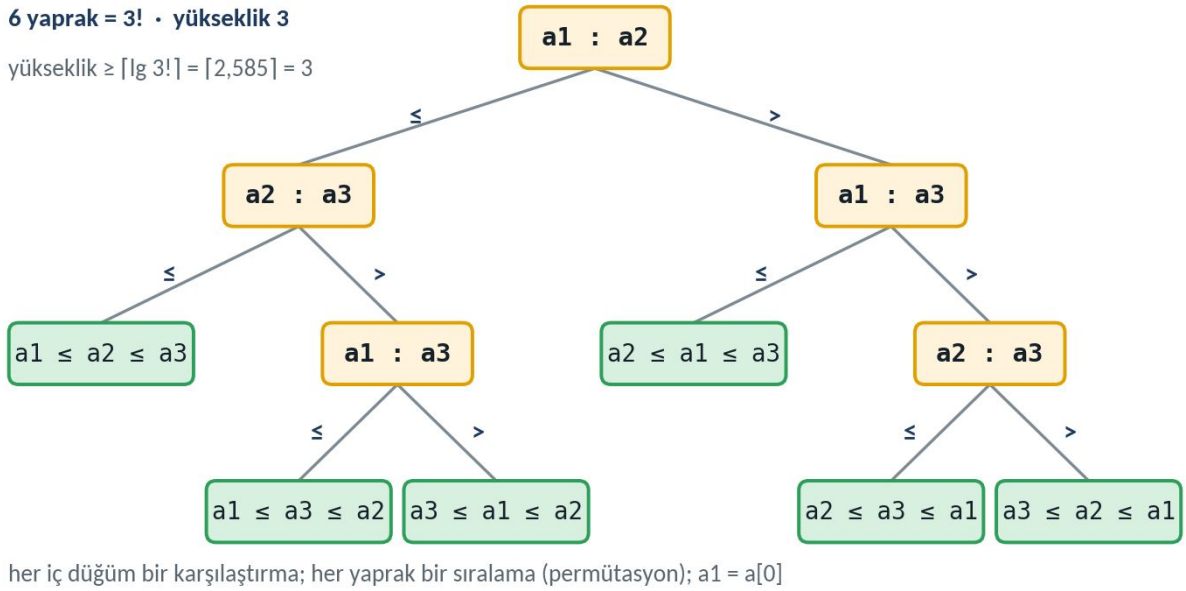


> NOT · **Ana teoremin sınırları (kapsam)**
>
> - Teorem yalnızca **aynı boyutlu a alt probleme** bölen bağıntılara uygulanır: T(n) = T(n − 1) + n yerine koyma ile, T(n) = T(n/3) + T(2n/3) + n özyineleme ağacıyla çözülür.
> - Genel ana teoremde (Cormen vd., 2022) birleştirme işi herhangi bir f(n) olabilir ve "kök baskın" durumu (orada 3. durum) ayrıca **düzenlilik koşulu** $a \cdot f(n/b) \le c \cdot f(n)$ ($c < 1$) ister; $f(n) = \Theta(n^d)$ iken bu koşul $c = a/b^d < 1$ ile kendiliğinden sağlanır. $T(n) = 2T(n/2) + n \cdot \lg n$ bu biçimin dışındadır. Genel biçim, kanıt ve Akra–Bazzi gibi genellemeler Algoritma Analizi dersinin konusudur (Bölüm 13).

3.5 Karar ağacı ve Ω(n log n) alt sınırı

Birleştirmeli sıralama her girdide Θ(n log n) karşılaştırma yapar. Daha hızlısı olabilir mi? **Karşılaştırmaya dayalı** bir sıralama (elemanlar hakkında yalnızca "a ≤ b mi?" sorusuyla bilgi edinen algoritma) bir **karar ağacı** (decision tree) olarak çizilebilir: iç düğümler karşılaştırma, yapraklar girdinin bir sıralamasıdır. n farklı elemanın n! sıralamasının her biri bir yaprakta bulunmalıdır; yüksekliği h olan ikili ağacın en çok 2^h yaprağı vardır. Öyleyse $2^h \ge n!$, yani **$h \ge \lg(n!)$**: en kötü durumda en az $\lceil \lg n! \rceil$ karşılaştırma gerekir.

Şekil 6.4
n = 3 için araya ekleme sıralamasının karar ağacı: 6 yaprak (3!), yükseklik 3 ≥ ⌈lg 6⌉ = ⌈2,585⌉



n! ≥ (n/2)^(n/2) olduğundan lg(n!) ≥ (n/2)·lg(n/2): **lg(n!) = Θ(n log n)**. Kavram 1 sayılarla gösterir: n = 1000'de lg(n!) ≈ 8529,4, n·lg n ≈ 9965,8; birleştirmeli sıralamanın en kötü durumu W(n) = n⌈lg n⌉ − 2^⌈lg n⌉ + 1 bu sınıra çok yakındır (n = 8: W = 17, ⌈lg 8!⌉ = 16). Sonuç: **karşılaştırmaya dayalı hiçbir sıralama en kötü durumda Ω(n log n)'den hızlı olamaz**; birleştirmeli sıralama (ve ortalamada hızlı sıralama) asimptotik olarak en iyidir. H7'de bu sınırı karşılaştırma yapmayan sıralamalarla aşacağız.

3.6 Kaputun Altında: tampon, yerinde bölümleme, yığın ve kütüphaneler

> KAPUTUN ALTINDA · **Tamponu bir kez ayır, bütün çağrılarla paylaş**
>
> Birleştirme yerinde yapılamaz: iki yarı yardımcı bir tamponda (`yrd`) birleştirilip geri kopyalanır. Tamponu her çağrıda `malloc` ile ayırmak gereksiz iş ve sızıntı riskidir: Hata Avcısı 2'de `malloc çağrısı = 999 · ayrılan = 39904 bayt · free = 0`, LeakSanitizer `AddressSanitizer: 39904 byte(s) leaked in 999 allocation(s).` bildirir. Örnek 2 tamponu **bir kez** ayırır ve özyinelemeli yardımcıya işaretçiyle verir. Her çağrıda ayıran (sızıntısız) sürüm n = 10^6'da 139,0 ms ↔ 128,5 ms (oran 1,08) sürdü; yinelenen ölçümlerde fark gürültü düzeyinde kaldı: maliyet küçük ama gereksizdir, asıl sorun sızıntıdır.

> KAPUTUN ALTINDA · **Yerinde bölümleme ve önbellek**
>
> Birleştirmeli sıralama her düzeyde her elemanı tampona ve geri kopyalar: Örnek 2'de n = 8 için 48 kopya (2·n·lg n); aynı dizide hızlı sıralama 10 takasla yetinir. Bölümleme diziyi **ardışık** tarar ve ek dizi kullanmaz; ardışık erişim önbellek satırlarını tam kullanır. Bu yüzden ortalama karşılaştırma sayısı daha fazla olsa da (≈ 1,39 n lg n ↔ ≈ n lg n) hızlı sıralama pratikte çoğu zaman daha hızlıdır: C ile n = 10^7'de birleştirmeli sıralama, üçün ortancası pivotlu hızlı sıralamanın 1,36 katı sürdü (Bölüm 6).

KAPUTUN ALTINDA · **Özyineleme derinliği, kuyruk çağrısı ve küçük parçaya önce özyineleme**

Pivot hep uçta seçilirse (sıralı girdi + ilk eleman) bölünmeler n − 1 | 0 olur ve iki çağrılı (naif) sürümün derinliği **n**'ye çıkar: büyük n'de yığın taşar (stack overflow). Hata Avcısı 8'de 256 KB'lık yığınla ters girdide n = 20 000 çalıştırması `Segmentation fault` ile çöker. Sıralı girdide -O2 ile çökmez, -O0 ile çöker: büyük parça **son** (kuyruk) çağrıdadır ve GCC -O2'de bu kuyruk çağrısını döngüye çevirir (kuyruk çağrısı eleme; tail call elimination). Bu bir derleyici eniyilemesidir, C standardının güvencesi değildir.

Algoritmik çözüm: **küçük parçaya özyineleme, büyük parçaya döngü**. Özyineleme yalnızca boyu en çok yarım olan parçaya yapılır; büyük parça aynı çerçevede döngünün sonraki turunda işlenir. Derinlik pivot ne kadar kötü olursa olsun ⌊lg n⌋ + 1'i aşmaz (Sedgewick, 1998); karşılaştırmalar değişmez, yalnızca yığın değişir: Örnek 6'da sıralı girdi + ilk eleman için ağaç yüksekliği 1000, yığın derinliği 2.

KAPUTUN ALTINDA · **Kütüphaneler hangi algoritmayı seçiyor?**

- **C++ `std::sort` (libstdc++, GCC 13):** introsort (Musser, 1997): pivot üçün ortancasıdır; bölümleme derinliği 2·⌊lg n⌋'yi aşan parça öbek sıralamasına (heap sort, H7) devredilir; 16 elemandan kısa parçalar en sonda tek bir araya ekleme geçişine bırakılır (`bits/stl_algo.h`: `_S_threshold = 16`). En kötü durum O(n log n)'dir; `std::sort` kararsızdır, kararlısı `std::stable_sort`'tur.
- **C `qsort`:** standart algoritmayı ve kararlılığı belirtmez. Bu makinedeki glibc 2.39, n = 1024 rastgele dizide karşılaştırıcıyı 8944 kez çağırdı: Örnek 2'deki birleştirmeli sıralamanın sayısıyla aynı; 200 denemede eşit anahtarların sırası bozulmadı (Kavram 3). Bu bir **gözlemdir**, güvence değildir.
- **Python `list.sort` / `sorted`:** Timsort; Tim Peters'ın birleştirmeli sıralama ile araya eklemeyi birleştiren, dizideki hazır sıralı koşuları (run) kullanan **kararlı** melez sıralaması. Python 3.11'den beri koşuların birleştirme sırası Powersort yöntemiyle belirlenir (University of Liverpool, 2022). Python'un yerleşik sıralaması hızlı sıralama **değildir**.

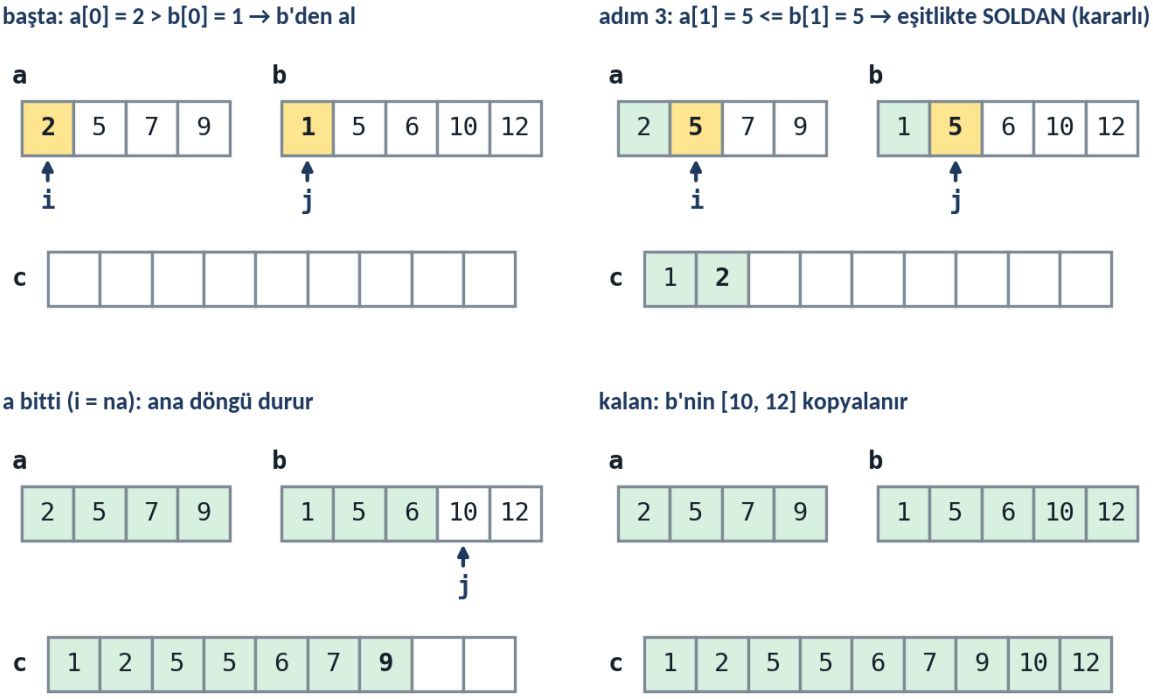
4. Algoritmalar

Beş amiral algoritmayı aynı on bir adımlı standart anlatımla işliyoruz: problem, fikir, elle izleme, sözde kod, doğruluk, analiz, C, üç dil, test, deney, varyantlar. İzlemelerde haftanın dizisi **[6, 3, 8, 1, 7, 2, 5, 4]** kullanılır; indisler 0 tabanlı, aralıklar kapalıdır: a[lo..hi].

4.1 Birleştirme ve birleştirmeli sıralama

1 · Problem. Girdi: a[0..n − 1]; çıktı: aynı elemanların artan sıralı düzeni; eşit anahtarlar özgün sıralarını korumalı (**kararlılık**). **2 · Fikir.** Diziyi ortadan ikiye böl, iki yarıyı aynı yöntemle sırala, iki sıralı yarıyı **birleştir** (merge). Birleştirme doğrusaldır: iki işaretçiden (i, j) küçüğü gösterilen eleman çıktıya yazılır ve o işaretçi ilerler; bir dizi bitince ötekinin kalanı karşılaştırmasız kopyalanır. Eşitlikte **soldan** almak (`<=`) kararlılığı sağlar (Şekil 6.5).

Şekil 6.5
Birleştirme adımı (Örnek 1): sarı, karşılaştırılan elemanlar; yeşil, c'ye yazılanlar. Adım 3'te 5 = 5 eşitliğinde soldaki alınır (kararlılık)



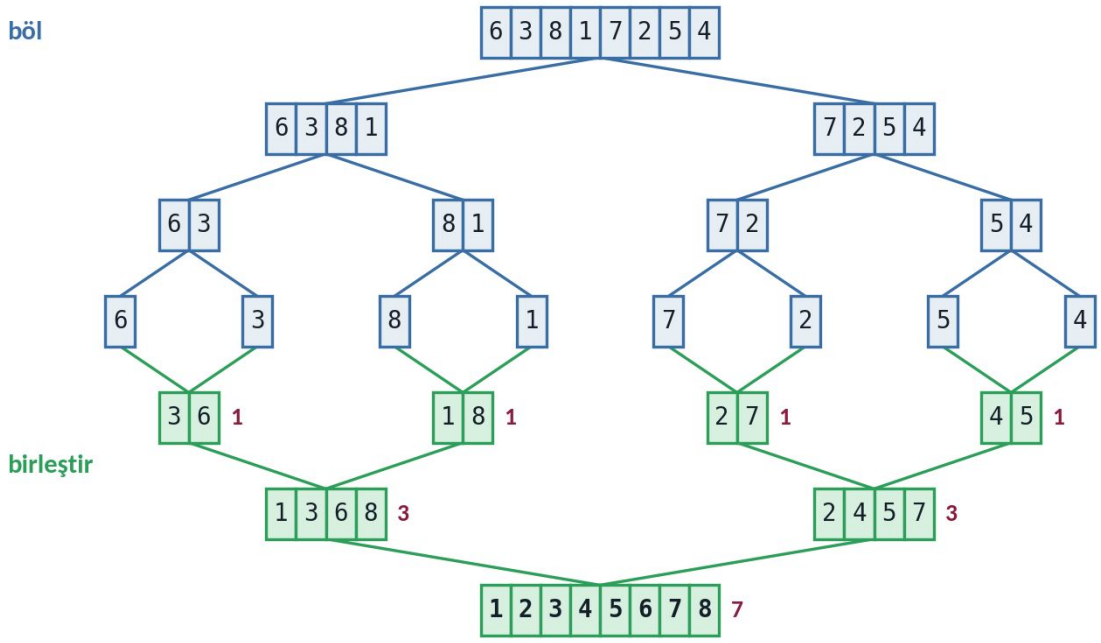
7 karşılaştırma (en çok na + nb − 1 = 8); değişmez: c[0..k−1] sıralı ve a[0..i−1] ∪ b[0..j−1]

3 · Elle izleme. Örnek 1, [2, 5, 7, 9] ile [1, 5, 6, 10, 12]'yi 7 karşılaştırmayla birleştirir (en çok na + nb − 1 = 8); a bitince b'nin kalanı [10, 12] kopyalanır. Birleştirmeli sıralamanın tamamı bir ağaçtır (Şekil 6.6): üstte bölünmeler, altta birleştirmeler; kırmızı sayıların toplamı Örnek 2'nin sayacıyla aynıdır.

SÖZDE KOD · 4 · Sözde kod: birleştirme ve birleştirmeli sıralama

```
algoritma BİRLEŞTİR(a, lo, mid, hi)        ▷ a[lo..mid] ve a[mid+1..hi] sıralı
    i ← lo;  j ← mid + 1;  k ← lo
    i ≤ mid ve j ≤ hi olduğu sürece
        eğer a[i] ≤ a[j] ise yrd[k] ← a[i]; i ← i + 1   ▷ eşitlikte soldan
        değilse yrd[k] ← a[j]; j ← j + 1
        k ← k + 1
    sol kalanı, sonra sağ kalanı yrd[k..]'ya kopyala;  a[lo..hi] ← yrd[lo..hi]
algoritma BİRLEŞTİRMELİ-SIRALA(a, lo, hi)
    eğer lo ≥ hi ise döndür                  ▷ 0 ya da 1 eleman: sıralı
    mid ← lo + ⌊(hi − lo) / 2⌋               ▷ böl
    BİRLEŞTİRMELİ-SIRALA(a, lo, mid)          ▷ fethet: sol yarı
    BİRLEŞTİRMELİ-SIRALA(a, mid + 1, hi)      ▷ fethet: sağ yarı
    BİRLEŞTİR(a, lo, mid, hi)                 ▷ birleştir
```

Şekil 6.6
[6, 3, 8, 1, 7, 2, 5, 4] için birleştirmeli sıralama ağacı: bölünme yukarıda (mavi), birleşme aşağıda (yeşil); 1 + 1 + 1 + 1 + 3 + 3 + 7 = 17 karşılaştırma



kırmızı sayılar: o birleştirmedeki karşılaştırma; toplam 17 (Örnek 2)

> TAHMİN ET · **Sizce sırada ne var?**
>
> Kök birleştirmesinde sol [1, 3, 6, 8], sağ [2, 4, 5, 7]'dir. Kaçıncı karşılaştırmadan sonra sol yarıda yalnızca 8 kalır? (Yanıt: 1, 2, 3, 4, 5, 6 alındıktan sonra, 6. karşılaştırmada; 7.'de 7 alınır ve 8 karşılaştırmasız kopyalanır.)

5 · Doğruluk. Birleştirme değişmezi: yrd[lo..k−1] sıralıdır ve a[lo..i−1] ile a[mid+1..j−1]'in tamamını içerir; her adımda kalanların en küçüğü eklendiği için korunur. Sıralama n üzerinde **tümevarımla** doğrudur: n ≤ 1'de dizi sıralıdır; n > 1'de iki yarı (boyları < n) sıralıdır ve birleştirme sıralı bir dizi üretir; mid = lo + ⌊(hi − lo)/2⌋ iki parçanın da küçüldüğünü güvenceye alır (Hata Avcısı 4). **6 · Analiz.** T(n) = 2T(n/2) + Θ(n) → **Θ(n log n)**, her girdide. Karşılaştırma ≤ W(n) = n⌈lg n⌉ − 2^⌈lg n⌉ + 1 (n = 1000: 8 724 ≤ 8 977); çağrı tam 2n − 1, derinlik ⌈lg n⌉ + 1; her düzeyde her eleman tampona gidip döner: 2·n·⌈lg n⌉ kopya (Tablo 6.2).

Tablo 6.2
Birleştirmeli sıralama kimlik kartı

ALGORİTMA KİMLİK KARTI	**Birleştirmeli sıralama** *(merge sort)*
Strateji	Böl ve fethet: diziyi ortadan ikiye böl, iki yarıyı özyinelemeyle sırala, iki sıralı yarıyı doğrusal zamanda birleştir
Girdi önkoşulu	Yok — elemanlar karşılaştırılabilir olmalı; n elemanlık ek tampon ayrılabilmeli
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(n) (tek yardımcı tampon) + O(log n) yığın

Özellikler	Kararlı (eşitlikte soldan al: <=) · yerinde değil · uyarlanabilir değil (sıralı girdide bile (n/2)·lg n karşılaştırma) · bağlı listeye ve dış bellek sıralamasına uygun. Karşılaştırma ≤ n⌈lg n⌉ − 2^⌈lg n⌉ + 1; çağrı 2n − 1; derinlik ⌈lg n⌉ + 1
Değişmez / doğruluk	birlestir: yrd[lo..k−1] sıralıdır ve a[lo..i−1] ile a[mid+1..j−1]'in tamamını içerir. bs(lo, hi) döndüğünde a[lo..hi] sıralıdır
Kütüphane karşılığı	C: yok (elle); `qsort` algoritma ve kararlılık garantisi vermez · C++: `std::stable_sort` (kararlı), `std::merge`, `std::inplace_merge` · Python: `sorted(a)`, `a.sort()` (Timsort: birleştirmeli + araya ekleme melezi, kararlı); `heapq.merge`
Ne zaman kullanılır	Kararlılık gerektiğinde (çok ölçütlü sıralama), en kötü durum Θ(n log n) garantisi istendiğinde, bağlı listelerde ve belleğe sığmayan veride (dış birleştirme)
Ne zaman kullanılmaz	Ek bellek kısıtlıysa (O(n) tampon); çok küçük dizilerde (araya ekleme daha hızlı: melez sürüm)

7 · C gerçekleştirimi. (Örnek 2; sayaçlar `static` değişkenlerdir). n elemanlık `yrd` tamponu `birlestirmeli_sirala`'da **bir kez** `malloc` ile ayrılır, yardımcıya işaretçiyle verilir, sonda bir kez `free` edilir (satır 68–75; Bölüm 3.6):

c/H06_02_birlestirmeli_siralama.c · satır 22–60

```c
static void birlestir(int *a, int *yrd, int lo, int mid, int hi)
{
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        karsilastirma++;
        if (a[i] <= a[j]) {  /* eşitse soldan */
            yrd[k++] = a[i++];
        } else {
            yrd[k++] = a[j++];
        }
    }
    while (i <= mid) {
        yrd[k++] = a[i++];
    }
    while (j <= hi) {
        yrd[k++] = a[j++];
    }
    for (k = lo; k <= hi; k++) {  /* geri kopyala */
        a[k] = yrd[k];
    }
    kopya += 2LL * (hi - lo + 1);
}

/* a[lo..hi]'yi sıralar; d: özyineleme derinliği (kök = 1) */
static void bs_ozyineli(int *a, int *yrd, int lo, int hi, int d)
{
    cagri++;
    if (d > en_derin) {
        en_derin = d;
    }
    if (lo >= hi) {  /* 0 ya da 1 eleman: sıralı */
        return;
    }
    int mid = lo + (hi - lo) / 2;
```

```
    /* böl ve fethet: iki yarıyı ayrı ayrı sırala */
    bs_ozyineli(a, yrd, lo, mid, d + 1);
    bs_ozyineli(a, yrd, mid + 1, hi, d + 1);
    birlestir(a, yrd, lo, mid, hi);  /* birleştir */
}
```

8 · Üç dil karşılaştırması. `birlestir` ve `bs_ozyineli` üç dilde satır satır aynıdır (Ek A.2); ayrışma tamponun yaşam süresindedir: C'de `malloc`/`free`, C++'ta kapsam bitince kendiliğinden bırakılan `std::vector` (RAII), Python'da çöp toplayıcının bıraktığı liste (hardal: C'den anlamca farklı satırlar; dil notları Ek A'da):

C++ H06_02_birlestirmeli_siralama.cpp · satır 69–78

```
void birlestirmeli_sirala(std::vector<int>& a)
{
    if (a.empty()) {
        return;
    }
    std::vector<int> yrd(a.size());
    tampon_ayirma++;
    int n = static_cast<int>(a.size());
    bs_ozyineli(a, yrd, 0, n - 1, 1);
}
```

Python H06_02_birlestirmeli_siralama.py · satır 59–65

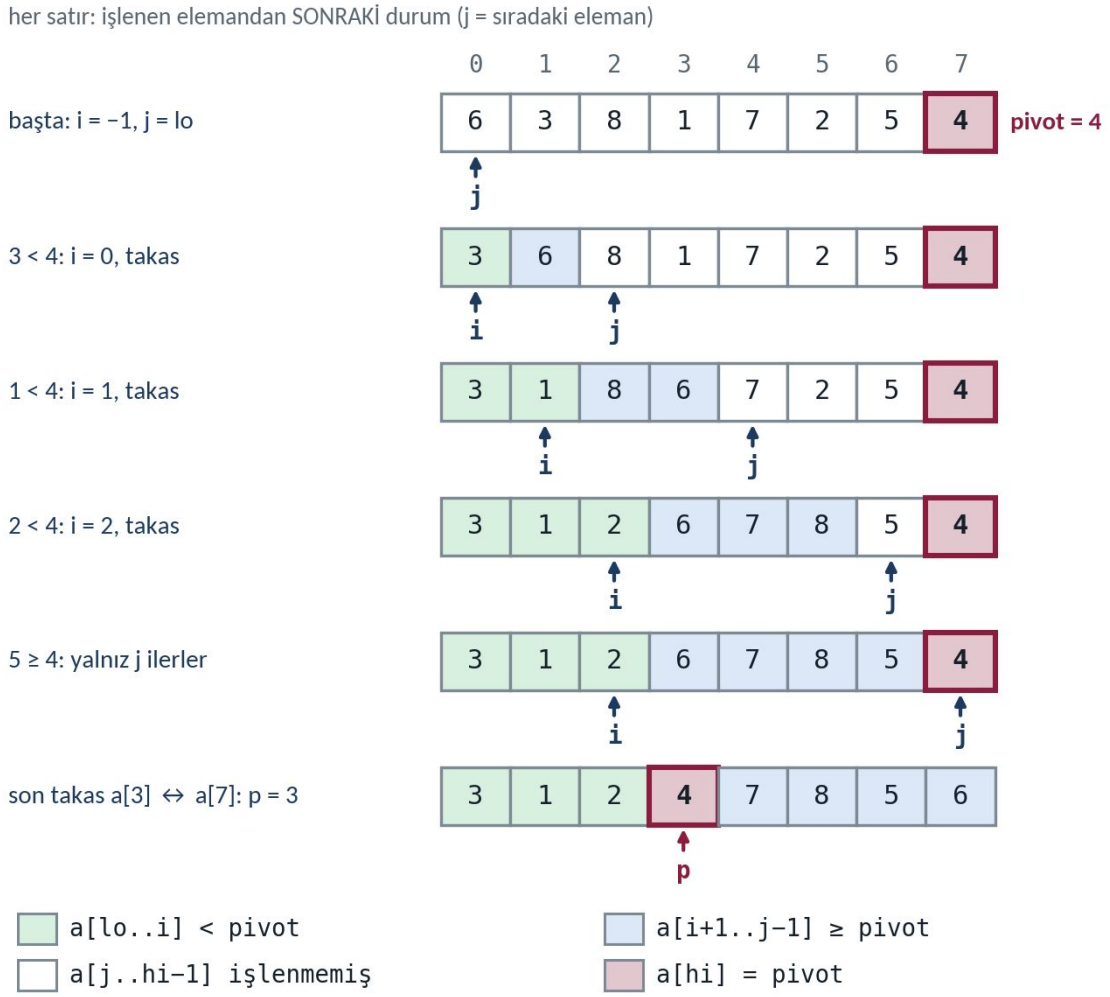
```
def birlestirmeli_sirala(a):
    global tampon_ayirma
    if not a:
        return
    yrd = [0] * len(a)
    tampon_ayirma += 1
    bs_ozyineli(a, yrd, 0, len(a) - 1, 1)
```

9 · Test. n = 1, 2, 3, 7, 8, 100 … 100 000 rastgele dizide sonuç kâhinle (`qsort`, `std::sort`, `sorted`) karşılaştırılır; çağrı = 2n − 1 ve derinlik = ⌈lg n⌉ + 1 denetlenir (n = 100 000: 1 536 273 ≤ W(n) = 1 568 929). Kararlılık testi Örnek 1'dedir: 70 puanlı Can, Ece (sol) ve Deniz (sağ) bu sırada çıkar. **10 · Deney.** Sayılar girdiden neredeyse bağımsızdır: n = 1024'te rastgele girdide 8 944, sıralı ve ters girdide 5 120 = (n/2)·lg n karşılaştırma, her durumda 20 480 kopya: algoritma **uyarlanabilir** (adaptive) değildir. C ile $n = 10^7$: 1,51 s (Bölüm 6). **11 · Varyantlar. Aşağıdan yukarıya** sürüm özyineleme kullanmaz (Örnek 10, Bölüm 5.1); **melez** sürüm küçük parçaları araya eklemeyle sıralar; **dış bellek birleştirmesi** belleğe sığmayan veriyi sıralı parçalar hâlinde diske yazıp k yollu birleştirir; Timsort hazır sıralı koşuları birleştirir. Kararlılık çok ölçütlü sıralamanın temelidir (Hata Avcısı 6–7).

4.2 Hızlı sıralama ve bölümleme

1 · Problem. Aynı sıralama problemi, bu kez **yerinde** (in-place): ek dizi olmadan, yalnızca takaslarla (swap). **2 · Fikir.** Bir **pivot** seç ve diziyi **bölümle** (partition): pivottan küçükler sola, diğerleri sağa, pivot araya, son yerine. Pivot bir daha yer değiştirmez; iki parça bağımsız olarak sıralanır, birleştirmeye gerek kalmaz. **Lomuto bölümlemesi** pivotu a[hi] alır ve diziyi tek bir j işaretçisiyle soldan sağa tarar; i, "küçükler" bölgesinin son elemanını gösterir (Şekil 6.7).

Şekil 6.7
Lomuto bölümleme şeridi (Örnek 4, pivot = a[hi] = 4): yeşil a[lo..i] < pivot, mavi a[i+1..j−1] ≥ pivot, beyaz işlenmemiş; her satır işlenen elemandan sonraki durum



3 · Elle izleme. j = 0 … 6 için a[j] pivotla karşılaştırılır; küçükse i artar ve a[i] ↔ a[j]. Sonda pivot a[i + 1]'e gider: p = 3, 7 karşılaştırma, 4 takas (Örnek 4'ün izleme tablosu aynı adımları yazar). Sol parça [3, 1, 2]'de pivot 2'dir: yalnızca 1 sola geçer ve 2 ortaya oturur.

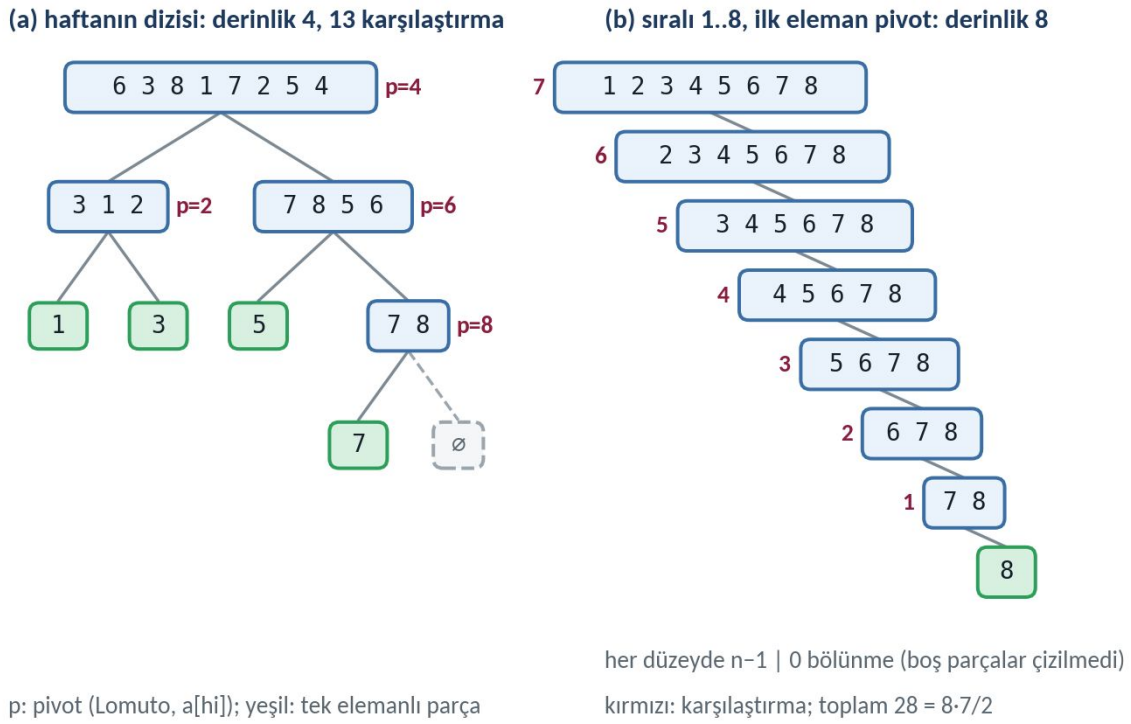
SÖZDE KOD · 4 · Sözde kod: Lomuto bölümleme ve hızlı sıralama

```
algoritma LOMUTO-BÖLÜMLE(a, lo, hi)          ▷ pivot = a[hi]
    pivot ← a[hi];  i ← lo − 1
    j ← lo'dan hi − 1'e kadar her j için
        eğer a[j] < pivot ise i ← i + 1;  a[i] ↔ a[j]   ▷ küçükler bölgesi büyür
    a[i + 1] ↔ a[hi];  döndür i + 1            ▷ pivot son yerinde
algoritma HIZLI-SIRALA(a, lo, hi)
    eğer lo ≥ hi ise döndür
    p ← LOMUTO-BÖLÜMLE(a, lo, hi)              ▷ böl (asıl iş)
    HIZLI-SIRALA(a, lo, p − 1);  HIZLI-SIRALA(a, p + 1, hi)   ▷ birleştirme yok
```

5 · Doğruluk. Değişmez: her turun başında **a[lo..i] < pivot ≤ a[i+1..j−1]**, a[j..hi−1] işlenmemiş. Başta iki bölge boştur; a[j] < pivot ise i'yi artırıp takas etmek küçükler bölgesini bir büyütür, değilse yalnızca j ilerler. j = hi'de son takas pivotu araya koyar: a[lo..p−1] < a[p] ≤ a[p+1..hi]; parçalar tümevarımla sıralanır. **6 · Analiz.** Bölümleme n − 1 karşılaştırma yapar. **En iyi durumda** pivot ortancadır:

$T(n) = 2T(n/2) + n \rightarrow \Theta(n \log n)$. **En kötü durumda** pivot hep uçtadır: $T(n) = T(n-1) + n - 1 \rightarrow n(n-1)/2 = \Theta(n^2)$; Lomuto için bu sıralı ve ters sıralı girdidir (n = 500: 124 750 karşılaştırma, derinlik 500; Şekil 6.8b).

Şekil 6.8
Hızlı sıralama ağaçları: (a) haftanın dizisinde dengeli bölünmeler (derinlik 4); (b) sıralı girdide ilk eleman pivotuyla zincir: her düzeyde n − 1 | 0 bölünme, n(n − 1)/2 karşılaştırma



Ortalama durum sezgisi. Rastgele girdide (ya da rastgele pivotla) i. ve j. en küçük elemanlar, aralarındaki j − i + 1 elemandan **ilk pivot seçilen** onlardan biriyse karşılaştırılır; yoksa ayrı parçalara düşer ve hiç karşılaşmazlar. Bu olasılık 2/(j − i + 1)'dir; bütün çiftler üzerinden toplam ≈ 2n·ln n ≈ **1,39·n·lg n** eder (Cormen vd., 2022; Sedgewick & Wayne, 2011); kesin değer $C(n) = 2(n+1)H(n) - 4n$'dir (Bölüm 13). Örnek 4'te n = 100 000: 1 982 734 karşılaştırma, beklenen 2 018 053 (oran 0,982); derinlik 38.

Tablo 6.3
Hızlı sıralama kimlik kartı

ALGORİTMA KİMLİK KARTI **Hızlı sıralama** *(quicksort)*	
Strateji	Böl ve fethet: pivot seç, diziyi pivotun çevresinde bölümle (Lomuto ya da Hoare), iki parçayı özyinelemeyle sırala; birleştirme adımı yoktur
Girdi önkoşulu	Yok — elemanlar karşılaştırılabilir olmalı
Zaman (en iyi · ortalama · en kötü)	$\Theta(n \log n)$ · $\Theta(n \log n)$ · $\Theta(n^2)$
Ek bellek	$O(\log n)$ yığın (ortalama; küçük parçaya özyineleme ile en kötü durumda da)
Özellikler	Kararsız · yerinde · pivot seçimine duyarlı: sıralı girdide ilk eleman pivotu n(n−1)/2 karşılaştırma ve n derinlik verir. Rastgele girdide beklenen karşılaştırma $2(n+1)H(n) - 4n \approx 1{,}39\, n \lg n$. Çok tekrar eden anahtarlarda üç yollu bölümleme gerekir

Değişmez / doğruluk	Lomuto: a[lo..i] < pivot ≤ a[i+1..j−1], a[j..hi−1] işlenmemiş, a[hi] = pivot. Bölümleme sonunda pivot son yerindedir
Kütüphane karşılığı	C: `qsort` (algoritma standartta belirtilmez) · C++: `std::sort` (libstdc++: introsort, kararsız); `std::partition` · Python: yok (`list.sort` Timsort'tur, hızlı sıralama değil)
Ne zaman kullanılır	Genel amaçlı bellek içi sıralama; ek bellek istenmediğinde; ortalama hızın önemli olduğu yerlerde (sıralı bellek erişimi, önbellek dostu bölümleme)
Ne zaman kullanılmaz	Kararlılık gerektiğinde; en kötü durum garantisi şartsa ya da girdi saldırgan olabilirse (McIlroy, 1999) — introsort ya da birleştirmeli sıralama

7 · C gerçekleştirimi. (Örnek 4; `takas` sayaçlı bir yardımcı, `d` yalnızca derinlik ölçümü için):

C `c/H06_04_lomuto_hizli.c · satır 28–55`

```c
int lomuto_bolumle(int *a, int lo, int hi)
{
    int pivot = a[hi];
    int i = lo - 1;
    for (int j = lo; j < hi; j++) {
        karsilastirma++;
        if (a[j] < pivot) {
            i++;
            takas(a, i, j);
        }
    }
    takas(a, i + 1, hi);  /* pivot son yerine */
    return i + 1;
}

/* d: özyineleme derinliği (kök = 1) */
void hizli_sirala(int *a, int lo, int hi, int d)
{
    if (d > en_derin) {
        en_derin = d;
    }
    if (lo >= hi) {
        return;
    }
    int p = lomuto_bolumle(a, lo, hi);
    hizli_sirala(a, lo, p - 1, d + 1);
    hizli_sirala(a, p + 1, hi, d + 1);
}
```

8 · Üç dil karşılaştırması. Tarama döngüsü üç dilde aynıdır; fark takasın yazımındadır:

C++ `H06_04_lomuto_hizli.cpp · satır 33–41`

```cpp
    for (int j = lo; j < hi; j++) {
        karsilastirma++;
        if (a[j] < pivot) {
            i++;
            takas(a, i, j);
        }
    }
    takas(a, i + 1, hi);  // pivot son yerine
    return i + 1;
```

Python `H06_04_lomuto_hizli.py · satır 27–33`

```python
    for j in range(lo, hi):
        karsilastirma += 1
        if a[j] < pivot:
            i += 1
            takas(a, i, j)
    takas(a, i + 1, hi)  # pivot son yerine
    return i + 1
```

9 · Test. Haftanın dizisi izlenir (13 karşılaştırma, 10 takas, derinlik 4); n = 10 … 100 000 rastgele dizide sonuç kâhinle, sayaç beklentiyle karşılaştırılır; en kötü durum testi sıralı ve ters sıralı n = 500 girdidir. **10 · Deney.** C ile $n = 10^7$ rastgele girdi: 1,13 s (ilk eleman), 1,20 s (rastgele pivot), 1,11 s (üçün ortancası); sıralı girdide ilk eleman pivotunun karesel çöküşü Bölüm 6.1'dedir. **11 · Varyantlar.** Hoare bölümlemesi, pivot seçimi ve üç yollu bölümleme aşağıda; melez sıralama Bölüm 5.2'de, introsort Bölüm 9'dadır.

Hoare bölümleme (Örnek 5)

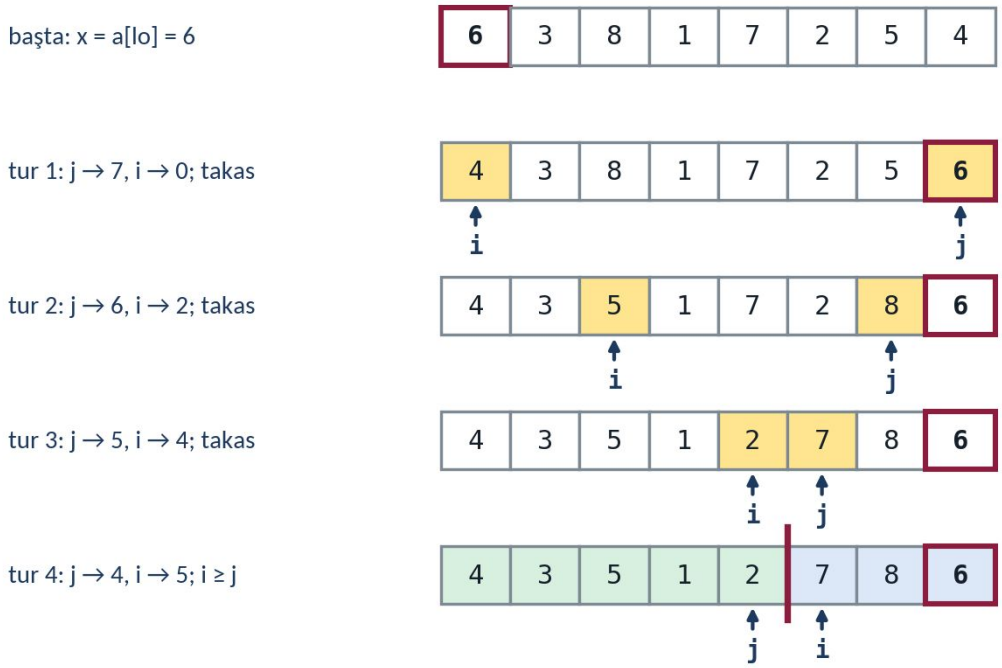
Hoare'un yöntemi diziyi iki uçtan tarar (Hoare, 1962). Burada kullandığımız biçimde (Cormen vd., 2022) x = a[lo]; j sağdan x'ten **büyük olmayan**, i soldan x'ten **küçük olmayan** ilk elemanda durur ve ikisi takas edilir; i ≥ j olunca j döndürülür. Haftanın dizisinde 3 takas ve 10 karşılaştırmadan sonra j = 4 döner (Şekil 6.9). **j pivotun yeri değildir**, yalnızca bir sınırdır: a[lo..j] ≤ x ≤ a[j+1..hi]; pivot 6 burada a[7]'dedir. Özyineleme bu yüzden [lo..j] ve [j+1..hi] üzerinde yapılır (Hata Avcısı 5).

C `c/H06_05_hoare_bolumleme.c · satır 28–46`

```c
int hoare_bolumle(int *a, int lo, int hi)
{
    int x = a[lo];
    int i = lo - 1, j = hi + 1;
    for (;;) {
        do {  /* sağdan: x'ten büyük olmayanı bul */
            j--;
            karsilastirma++;
        } while (a[j] > x);
        do {  /* soldan: x'ten küçük olmayanı bul */
            i++;
            karsilastirma++;
        } while (a[i] < x);
        if (i >= j) {
            return j;
```

```
43          }
44          takas(a, i, j);
45      }
46  }
```

Şekil 6.9
Hoare bölümleme (Örnek 5, x = a[lo] = 6): sarı, takas edilen çift; kırmızı çizgi dönüş sınırı j = 4. Pivot 6, a[4]'te değil a[7]'dedir



dönüş j = 4: a[0..4] ≤ 6 ≤ a[5..7]; pivot 6 a[7]'de, a[4]'te DEĞİL

Hoare: 10 karşılaştırma, 3 takas · özyineleme [lo..j] ve [j+1..hi] (Örnek 5)

Hoare yalnızca iki eleman da yanlış taraftaysa takas eder: iki uçtan tarayan bölümlemeyle rastgele girdide ortalama ≈ (1/3)·n ln n takas yapılır (Sedgewick & Wayne, 2011); Lomuto ise pivottan küçük her elemanı takas eder. Örnek 5'te n = 100 000 için Hoare 388 046 (formül ≈ 383 764), Lomuto 1 098 656 takas yapar; Lomuto/Hoare oranı n = 100'de 2,19, n = 100 000'de 2,83. Karşılaştırma ise Hoare'da biraz fazladır (2 572 445 ↔ 1 982 734). Asıl fark **eşit anahtarlardadır**: n = 500 tümü eşit girdide Lomuto hiçbir elemanı sola almaz (124 750 karşılaştırma, derinlik 500); Hoare her eşitte durup takas eder ve ortadan böler (5 430, derinlik 10). Python'da `do … while` yoktur: önce bir adım atılır, sonra `while` sınanır (Ek A.5).

Pivot seçimi ve özyineleme derinliği (Örnek 6)

En kötü durumun en sık nedeni **sıralı ya da neredeyse sıralı girdi** ile **ilk (son) eleman pivotudur**. Örnek 6 üç stratejiyi karşılaştırır: **ilk eleman**, **rastgele eleman** (ortak LCG, tohum 2026) ve **üçün ortancası** (a[lo], a[mid], a[hi]'nin ortancası; Sedgewick, 1998). Seçilen pivot a[hi]'ye taşınır, sonra Lomuto çalışır:

c/H06_06_pivot_deneyi.c · satır 56–65

```c
static int pivot_sec(const int *a, int lo, int hi, int strateji)
{
    if (strateji == ILK) {
        return lo;
    }
    if (strateji == RASTGELE) {
        return rastgele_aralik(lo, hi);
    }
    return ortanca3(a, lo, lo + (hi - lo) / 2, hi);
}
```

n = 1000 sıralı girdide ilk eleman 499 500 karşılaştırma yapar (= n(n − 1)/2), rastgele pivot 10 898, üçün ortancası 9 009. **Oran testi** (H2) n iki katına çıkınca Θ(n²) için 4, Θ(n log n) için ≈ 2,20 bekler: sıralı girdide ilk eleman 4,00, üçün ortancası 2,22 verir. Üçün ortancası ters girdide rastgele pivottan kötüdür (15 773 ↔ 10 224) ve kurgulanmış girdilerle karesel davranışa zorlanabilir (Atölye 2); rastgele pivotu hiçbir **sabit** girdi tetikleyemez. Örnek 6 ayrıca **küçük parçaya özyineleme, büyüğe döngü** uygular (kod kalıbı Örnek 11'de, Bölüm 5.2): ağaç yüksekliği 1000 olan sıralı/ilk eleman çalıştırmasında yığın derinliği 2'dir.

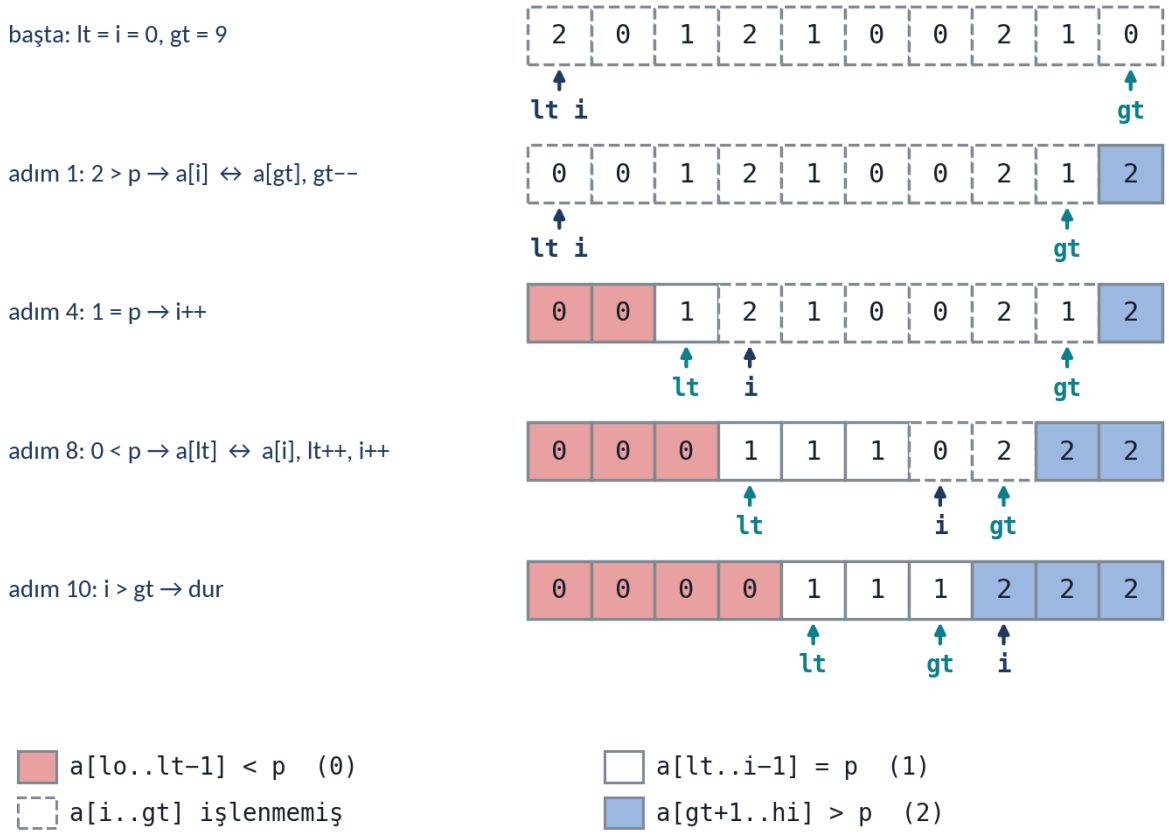
Eşit anahtarlar ve üç yollu bölümleme (Örnek 7)

Az sayıda farklı anahtar varsa (notlar, il kodları) iki yollu bölümleme eşitleri hep aynı tarafa gönderir. Dijkstra'nın **Hollanda bayrağı** problemi çözümü verir: üç bölge (< p, = p, > p) ve üç işaretçi (lt, i, gt) (Dijkstra, 1976). a[i] < p ise a[lt] ile takas edilip lt ve i artar; a[i] > p ise a[gt] ile takas edilip yalnızca gt azalır (gelen eleman henüz incelenmedi); eşitse yalnızca i artar (Şekil 6.10). Örnek 7'de on topluk bayrak tek geçişte 16 karşılaştırma ve 7 takasla sıralanır; a[lt..gt] = p bölgesi bir daha işlenmez (Bentley & McIlroy, 1993).

c/H06_07_uc_yollu_bolumleme.c · satır 31–52

```c
void uc_yollu_bolumle(int *a, int lo, int hi, int p, int *lt_c, int *gt_c)
{
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        karsilastirma++;
        if (a[i] < p) {  /* küçük: sola */
            takas(a, lt, i);
            lt++;
            i++;
        } else {
            karsilastirma++;
            if (a[i] > p) {  /* büyük: sağa */
                takas(a, i, gt);
                gt--;
            } else {  /* eşit: yerinde kalır */
                i++;
            }
        }
    }
    *lt_c = lt;
    *gt_c = gt;
}
```

Şekil 6.10
Üç yollu bölümleme (Örnek 7, p = 1): kırmızı a[lo..lt−1] < p, beyaz a[lt..i−1] = p, kesikli işlenmemiş, mavi a[gt+1..hi] > p



n = 2000'de yalnızca 0 ve 1'den oluşan girdide iki yollu bölümleme 1 001 099 karşılaştırma yapar (ağaç yüksekliği 1011), üç yollu 5 980 (yükseklik 3). Bedeli: hepsi farklı girdide eleman başına iki karşılaştırma yapabildiğinden daha pahalıdır (44 007 ↔ 24 692). İki sınır döndürmek için C'de çıkış parametreleri, C++17'de `auto [lt, gt]`, Python'da demet kullanılır (Ek A.7).

4.3 Ters çevrim sayısı

1 · Problem. Ters çevrim (inversion) sayısı, i < j ve a[i] > a[j] olan çiftlerin sayısıdır: sıralı dizide 0, ters sıralıda n(n − 1)/2. Dizinin "ne kadar sırasız" olduğunu ölçer: araya ekleme tam bu kadar kaydırma yapar (H5); iki sıralamanın benzerliği (öneri sistemleri) de bu sayıya dayanır (Kleinberg & Tardos, 2006). Kaba yöntem bütün çiftleri dener: $\Theta(n^2)$. **2 · Fikir.** Ters çevrimler ya sol yarıda, ya sağ yarıda, ya da **kesişendir** (biri solda, biri sağda). Kesişenleri saymak, iki yarı **sıralıyken** kolaydır: birleştirmede sağdan a[j] alındığında solda kalan a[i..mid]'nin hepsi a[j]'den büyüktür: mid − i + 1 ters çevrim. Sayım birleştirmenin içine bedavaya yerleşir.

3 · Elle izleme. Sol yarı [6, 3, 8, 1] 4, sağ yarı [7, 2, 5, 4] 4 ters çevrim taşır; kökte 2 alınırken solda 3, 4'te 2, 5'te 2, 7'de 1 eleman kalmıştır: 8 kesişen. Toplam 16, kaba yöntemle aynı. **4 · Sözde kod.** BİRLEŞTİR'in "değilse" dalına `sayı ← sayı + (mid − i + 1)` eklenir; TERS-ÇEVRİM(lo, hi) = TERS-ÇEVRİM(lo, mid) + TERS-ÇEVRİM(mid + 1, hi) + BİRLEŞTİR-SAY(lo, mid, hi), lo ≥ hi ise 0.

5 · Doğruluk. Değişmez: birlestir_say döndüğünde sayi, bir eleman sol yarıdan, diğeri sağ yarıdan olan ters çevrim çiftlerinin sayısıdır; toplam = sol + sağ + kesişen. Her çift tam bir kez sayılır: aynı yarıdaysa o yarının çağrısında, kesişense kökte, sağdaki alınırken. Eşitlikte soldan almak (`<=`) şarttır: a[i] = a[j] ters

çevrim değildir. **6 · Analiz.** T(n) = 2T(n/2) + n → Θ(n log n); karşılaştırmalar birleştirmeli sıralamanınkiyle aynıdır. Sayaç `long long` olmalıdır: n = 100 000 ters sıralı dizide 4 999 950 000 > 2^{31} − 1 (Tablo 6.4).

Tablo 6.4
Ters çevrim sayma kimlik kartı

ALGORİTMA KİMLİK KARTI **Ters çevrim sayma (birleştirmeli)** *(counting inversions (merge-sort based))*	
Strateji	Böl ve fethet: birleştirme adımını genişletme — sağdan eleman alınınca soldaki kalan mid − i + 1 elemanın her biri bir ters çevrimdir
Girdi önkoşulu	Yok — elemanlar karşılaştırılabilir olmalı
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(n)
Özellikler	Sayı n(n−1)/2'ye kadar çıkar (C/C++'ta long long). Girdi bozulmasın diye kopya üzerinde çalışılır. Sonuç, araya ekleme sıralamasının kaydırma sayısına eşittir (H5); kâhin Θ(n^2) çift taraması
Değişmez / doğruluk	birlestir_say döndüğünde sayi, bir eleman sol yarıdan, diğeri sağ yarıdan olan ters çevrim çiftlerinin sayısıdır; toplam = sol + sağ + kesişen
Kütüphane karşılığı	C: yok · C++: yok · Python: yok
Ne zaman kullanılır	İki sıralama arasındaki uzaklık (Kendall tau), benzerlik ölçümü (öneri sistemleri), bir dizinin ne kadar sırasız olduğunun ölçüsü
Ne zaman kullanılmaz	n küçükse (birkaç bin) Θ(n^2) çift taraması yeterince hızlı ve daha basittir

7 · C gerçekleştirimi. (Örnek 8; girdiyi bozmamak için kopya üzerinde sayılır, sayım sıralamanın yan ürünüdür):

c/H06_08_ters_cevrim.c · satır 20–55

```c
static long long birlestir_say(int *a, int *yrd, int lo, int mid, int hi)
{
    long long sayi = 0;
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        karsilastirma++;
        if (a[i] <= a[j]) {  /* eşitse sayılmaz */
            yrd[k++] = a[i++];
        } else {
            yrd[k++] = a[j++];
            sayi += mid - i + 1;  /* hepsi > a[j] */
        }
    }
    while (i <= mid) {
        yrd[k++] = a[i++];
    }
    while (j <= hi) {
        yrd[k++] = a[j++];
    }
    for (k = lo; k <= hi; k++) {
        a[k] = yrd[k];
    }
```

```
    return sayi;
}

/* a[lo..hi] içindeki ters çevrim sayısı; a sıralanmış olarak kalır */
static long long ters_cevrim_bs(int *a, int *yrd, int lo, int hi)
{
    if (lo >= hi) {
        return 0;
    }
    int mid = lo + (hi - lo) / 2;
    long long sol = ters_cevrim_bs(a, yrd, lo, mid);
    long long sag = ters_cevrim_bs(a, yrd, mid + 1, hi);
    return sol + sag + birlestir_say(a, yrd, lo, mid, hi);
}
```

8 · Üç dil karşılaştırması. Sayımın yapıldığı döngü:

C++ **H06_08_ters_cevrim.cpp · satır 26–34**

```
    while (i <= mid && j <= hi) {
        karsilastirma++;
        if (a[i] <= a[j]) {  // eşitse sayılmaz
            yrd[k++] = a[i++];
        } else {
            yrd[k++] = a[j++];
            sayi += mid - i + 1;  // hepsi > a[j]
        }
    }
```

Python **H06_08_ters_cevrim.py · satır 21–30**

```
    while i <= mid and j <= hi:
        karsilastirma += 1
        if a[i] <= a[j]:  # eşitse sayılmaz
            yrd[k] = a[i]
            i += 1
        else:
            yrd[k] = a[j]
            j += 1
            sayi += mid - i + 1  # hepsi > a[j]
        k += 1
```

9 · Test. Kâhin H5'in kaba çift taramasıdır: n = 10 … 3000 rastgele dizide sayılar aynı (n = 1000: 238 849 ters çevrim; 8 716 ↔ 499 500 karşılaştırma); sınır durumları sıralı (0), ters sıralı ve tek eleman. **10 · Deney.** n = 100 000 rastgele dizide 2 494 795 927 ters çevrim 1 536 273 karşılaştırmayla sayıldı; kaba yöntem 4 999 950 000 karşılaştırma isterdi. **11 · Varyantlar.** Aynı fikir “a[i] > 2·a[j] çiftleri” gibi sorulara uyarlanır; Fenwick ağacıyla sayım da Θ(n log n)'dir.

4.4 Quickselect: k'ıncı en küçük ve ortanca

1 · Problem. Sırasız n elemanda k'ıncı en küçüğü (0 tabanlı) bul; k = ⌊n/2⌋ ortancadır (median). Sıralayıp a[k]'yı almak Θ(n log n)'dir. **2 · Fikir.** Bölümle, ama yalnızca **k'yı içeren parçaya** devam et (Hoare, 1961b). Örnek 9 üç yollu bölümleme kullanır: k eşitler bölgesine [lt..gt] düşerse yanıt pivottur; tek parçaya devam edildiği için bir döngü yeter.

3 · Elle izleme. Notlar 72, 45, 88, 61, 95, 53, 77, 61, 84; k = 4. Tur 1: [0..8], pivot 84, eşitler [6..6] → sola; Tur 2: [0..5], pivot 77, eşitler [5..5] → sola; Tur 3: [0..4], pivot 61, eşitler [2..3] → sağa; lo = hi = 4'te yanıt **72**.

SÖZDE KOD · 4 · Sözde kod: quickselect (yinelemeli)

```
algoritma HIZLI-SEÇ(a, k)                    ▷ k'ıncı en küçük (0 tabanlı)
    lo ← 0;  hi ← n − 1
    lo < hi olduğu sürece
        p ← a[RASTGELE(lo, hi)];  (lt, gt) ← ÜÇ-YOLLU-BÖLÜMLE(a, lo, hi, p)
        eğer k < lt ise hi ← lt − 1           ▷ yalnızca sol parça
        değilse eğer k > gt ise lo ← gt + 1   ▷ yalnızca sağ parça
        değilse döndür p                      ▷ k eşitler bölgesinde
    döndür a[lo]
```

5 · Doğruluk. Değişmez: Aranan k'ıncı eleman her turda a[lo..hi] içindedir: a[0..lo−1] ≤ a[lo..hi] ≤ a[hi+1..n−1]; aralık her turda en az eşitler bölgesi kadar küçüldüğünden döngü sonlanır. **6 · Analiz.** İyi pivot aralığı yarıya indirir: T(n) = T(n/2) + n → n + n/2 + … < 2n = **Θ(n)** (ana teorem: $a = 1 < b^d = 2$). Rastgele pivotla beklenen iş doğrusaldır; en kötü durumda T(n) = T(n − 1) + n → Θ(n^2) (Cormen vd., 2022). Ölçüm: n 1000 kat büyürken karşılaştırma/n tekrarlı notlarda 4,24, 4,90, 3,89, 4,74, farklı değerlerde 4,83, 5,04, 5,75, 4,92; lg n ise 6,66'dan 16,61'e çıkar.

Tablo 6.5
Quickselect kimlik kartı

ALGORİTMA KİMLİK KARTI	**Quickselect (hızlı seçim)** *(quickselect (Hoare's FIND))*
Strateji	Böl ve fethet (tek alt problem): bölümle, yalnızca k'nın bulunduğu parçada devam et
Girdi önkoşulu	0 ≤ k < n; dizi yeniden düzenlenebilir olmalı
Zaman (en iyi · ortalama · en kötü)	Θ(n) · Θ(n) · Θ(n^2)
Ek bellek	O(1) (yinelemeli)
Özellikler	k'ıncı en küçük, ortanca, yüzdelik. Rastgele pivotla beklenen iş doğrusal: karşılaştırma/n, n 1000 kat büyürken artmaz (Örnek 9'da üç yollu bölümlemeyle ≈ 4–6). Tekrarlı anahtarlarda (notlar) üç yollu bölümleme
Değişmez / doğruluk	Aranan k'ıncı eleman her turda a[lo..hi] içindedir: a[0..lo−1] ≤ a[lo..hi] ≤ a[hi+1..n−1]
Kütüphane karşılığı	C: yok (elle) · C++: `std::nth_element` (introselect) · Python: `statistics.median` (sıralar: O(n log n)), `heapq.nsmallest(k, a)`
Ne zaman kullanılır	Tek bir sıra istatistiği gerektiğinde: ortanca not, yüzdelik, en küçük k eleman
Ne zaman kullanılmaz	Aynı dizide çok sayıda sıra istatistiği gerekiyorsa (bir kez sıralamak daha ucuz); en kötü durum garantisi şartsa (medyanların medyanı — Algoritma Analizi dersi)

7 · C gerçekleştirimi. (Örnek 9; `uc_yollu_bolumle` Örnek 7'dekiyle aynıdır):

C `c/H06_09_quickselect.c · satır 51–68`

```c
int hizli_sec(int *a, int n, int k)
{
    int lo = 0, hi = n - 1;
    while (lo < hi) {
        tur_sayisi++;
        int p = a[rastgele_aralik(lo, hi)];
        int lt, gt;
        uc_yollu_bolumle(a, lo, hi, p, &lt, &gt);
        if (k < lt) {
            hi = lt - 1;  /* k soldaki parçada */
        } else if (k > gt) {
            lo = gt + 1;  /* k sağdaki parçada */
        } else {
            return p;  /* k, a[lt..gt] içinde */
        }
    }
    return a[lo];
}
```

8 · Üç dil karşılaştırması. İki sınır C++'ta yapılandırılmış bağlamayla, Python'da demetle alınır:

C++ `H06_09_quickselect.cpp · satır 49–61`

```cpp
    while (lo < hi) {
        tur_sayisi++;
        int p = a[rastgele_aralik(lo, hi)];
        auto [lt, gt] = uc_yollu_bolumle(a, lo, hi, p);
        if (k < lt) {
            hi = lt - 1;  // k soldaki parçada
        } else if (k > gt) {
            lo = gt + 1;  // k sağdaki parçada
        } else {
            return p;  // k, a[lt..gt] içinde
        }
    }
    return a[lo];
```

Python `H06_09_quickselect.py · satır 43–53`

```python
    while lo < hi:
        tur_sayisi += 1
        p = a[rastgele_aralik(lo, hi)]
        lt, gt = uc_yollu_bolumle(a, lo, hi, p)
        if k < lt:
            hi = lt - 1  # k soldaki parçada
        elif k > gt:
            lo = gt + 1  # k sağdaki parçada
        else:
            return p  # k, a[lt..gt] içinde
    return a[lo]
```

9 · Test. Kâhin kopyayı sıralayıp a[k]'ya bakmaktır; n = 101 … 100 001, beşer çalıştırma, iki girdi ailesi; sınır durumları tek eleman, k = 0, k = n − 1 ve tümü eşit girdi. **10 · Deney.** Mini-OBS 2 v2.5'te 10^6 ortalamanın ortancası quickselect ile 22,0 ms, sıralamayla 208,2 ms sürdü (Bölüm 10). **11 · Varyantlar.** C++ `std::nth_element` aynı işi yapar; pivot **medyanların medyanı** ile seçilirse en kötü durum da Θ(n)'dir (Blum vd., 1973; Bölüm 13).

4.5 En büyük alt dizi toplamı (böl ve fethet)

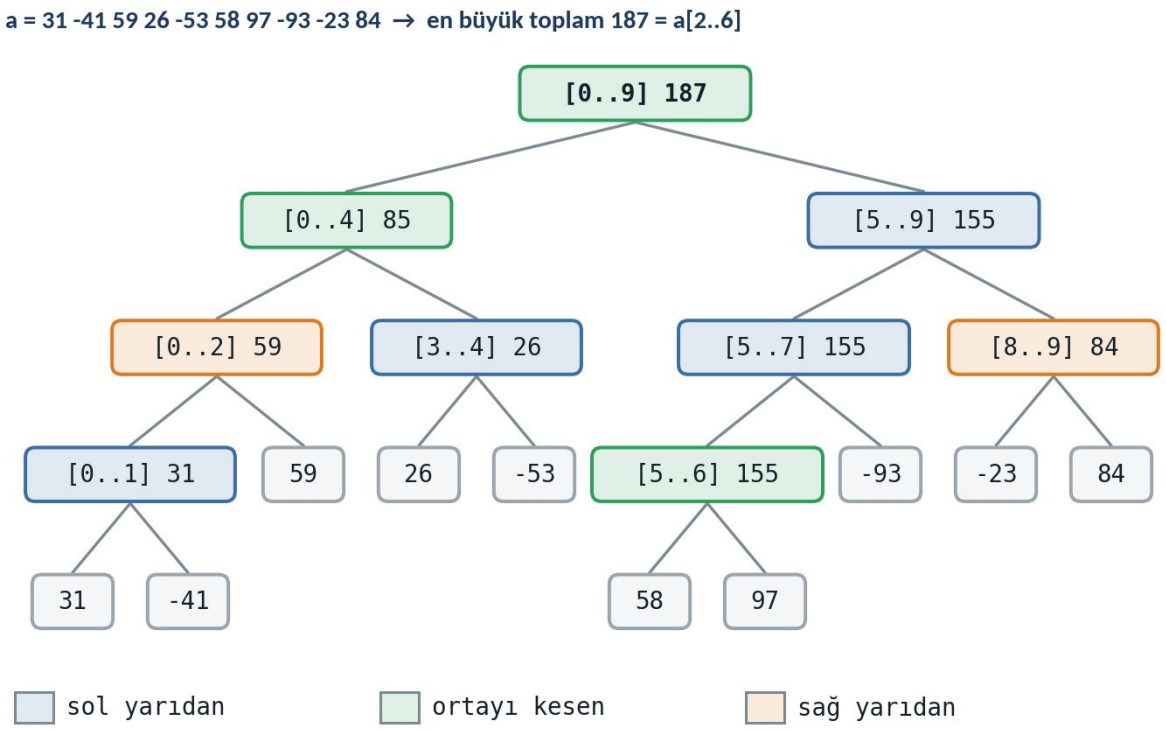
1 · Problem. H2'nin problemi: toplamı en büyük **ardışık** alt dizi (en az bir eleman); H2'de Θ(n^3), Θ(n^2) ve Kadane'nin Θ(n) çözümünü görmüştük. **2 · Fikir.** En iyi alt dizi ya tamamen sol yarıda, ya tamamen sağ yarıda, ya da ortayı **keser**. İlk ikisi özyinelemedir; ortayı keseni bulmak doğrusaldır: mid'den sola en iyi toplam + mid + 1'den sağa en iyi toplam. Sonuç üç adayın en büyüğüdür (Şekil 6.11).

3 · Elle izleme. Bentley'nin dizisinde [31, -41, 59, 26, -53, 58, 97, -93, -23, 84] sol yarının en iyisi 85 (a[2..3]), sağınki 155 (a[5..6]); ortayı kesen a[2..6] = 187 kazanır.

SÖZDE KOD · 4 · Sözde kod: en büyük alt dizi

```
algoritma EN-BÜYÜK(a, lo, hi)                ▷ (toplam, baş, son)
    eğer lo = hi ise döndür (a[lo], lo, lo)
    mid ← lo + ⌊(hi − lo) / 2⌋
    sol ← EN-BÜYÜK(a, lo, mid);  sağ ← EN-BÜYÜK(a, mid + 1, hi)
    orta ← KESİŞEN(a, lo, mid, hi)            ▷ mid'den sola + mid+1'den sağa, Θ(n)
    döndür sol, orta, sağ içinde toplamı en büyük olanı   ▷ eşitlikte bu sırayla
```

Şekil 6.11
Bentley'nin dizisinde böl ve fethet ağacı (Örnek 13): her düğümde aralığın en iyi toplamı; renk, sonucun hangi adaydan geldiğini gösterir



5 · Doğruluk. Her alt dizi üç sınıftan tam birindedir; KESİŞEN, mid'i içeren alt dizilerin sol ve sağ kısımlarını bağımsız en iyiler, çünkü toplam iki kısmın toplamıdır; tümevarımla sonuç doğrudur.

6 · Analiz. T(n) = 2T(n/2) + n → **Θ(n log n)**. Toplama sayısı birleştirmeli sıralamanın W(n)'sine birebir eşittir, çünkü her düğüm boyunun bir eksiği kadar toplama yapar (n = 1000: 8 977 = W(n); Kadane 999); yığın derinliği ⌈lg n⌉ + 1.

Tablo 6.6
En büyük alt dizi (böl ve fethet) kimlik kartı

ALGORİTMA KİMLİK KARTI **En büyük alt dizi (böl ve fethet)** *(maximum subarray (divide and conquer))*	
Strateji	Böl ve fethet: en iyi alt dizi ya sol yarıda, ya sağ yarıda ya da ortayı keser; kesişen en iyi parça Θ(n) taramayla bulunur
Girdi önkoşulu	n ≥ 1; alt dizi boş olamaz (H2 sözleşmesi)
Zaman (en iyi · ortalama · en kötü)	Θ(n log n) · Θ(n log n) · Θ(n log n)
Ek bellek	O(log n)
Özellikler	T(n) = 2T(n/2) + Θ(n). Toplama sayısı tam olarak n⌈lg n⌉ − 2^⌈lg n⌉ + 1 (birleştirmeli sıralamanın karşılaştırma sınırıyla aynı). H2 Kadane ile aynı toplam; Kadane Θ(n)
Değişmez / doğruluk	en_buyuk_bf(lo, hi), a[lo..hi]'nin boş olmayan alt dizileri arasında en büyük toplamı döndürür (tümevarım: sol, sağ, kesişen)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`itertools.accumulate` yalnızca önek toplamı)
Ne zaman kullanılır	Böl ve fethet stratejisini öğrenmek; parçalara bölünmüş veride (paralel/dağıtık) parça sonuçlarını birleştirmek
Ne zaman kullanılmaz	Tek dizide sonuç istendiğinde: Kadane Θ(n) ve O(1) bellekle daha hızlıdır (H2)

7 · C gerçekleştirimi. (Örnek 13; `kesisen` iki döngüdür, Ek A.13):

C **c/H06_13_en_buyuk_alt_dizi.c · satır 57–78**

```c
Aralik en_buyuk_bf(const int *a, int lo, int hi, int d)
{
    if (d > en_derin) {
        en_derin = d;
    }
    if (lo == hi) {
        Aralik tek = {a[lo], lo, lo};
        return tek;
    }
    int mid = lo + (hi - lo) / 2;
    Aralik sol = en_buyuk_bf(a, lo, mid, d + 1);
    Aralik sag = en_buyuk_bf(a, mid + 1, hi, d + 1);
    Aralik orta = kesisen(a, lo, mid, hi);
    Aralik en = sol;  /* eşitlikte sol, kesişen, sağ */
    if (orta.toplam > en.toplam) {
        en = orta;
    }
    if (sag.toplam > en.toplam) {
        en = sag;
    }
    return en;
}
```

8 · Üç dil karşılaştırması. Sonuç C'de yapı, C++'ta küme başlatmalı yapı, Python'da demettir:

C++ `H06_13_en_buyuk_alt_dizi.cpp · satır 68–76`

```cpp
    Aralik orta = kesisen(a, lo, mid, hi);
    Aralik en = sol;  // eşitlikte sol, kesişen, sağ
    if (orta.toplam > en.toplam) {
        en = orta;
    }
    if (sag.toplam > en.toplam) {
        en = sag;
    }
    return en;
```

Python `H06_13_en_buyuk_alt_dizi.py · satır 51–57`

```python
    orta = kesisen(a, lo, mid, hi)
    en = sol  # eşitlikte sol, kesişen, sağ
    if orta[0] > en[0]:
        en = orta
    if sag[0] > en[0]:
        en = sag
    return en
```

9 · Test. Kâhin H2'nin Kadane'sidir; tümü negatif dizi (sonuç en büyük eleman: −1 = a[1..1]), tek eleman, sıfırlar ve n = 10 … 100 000 rastgele dizi. **10 · Deney.** Toplama oranı (böl-fethet / Kadane) n = 10'da 2,8, n = 100 000'de 15,7: lg n gibi büyür. **11 · Varyantlar.** En yakın nokta çifti ve Karatsuba çarpımı (Alıştırma 6.14) aynı "iki yarı + doğrusal birleştirme" iskeletini kullanır.

5. Çözümlü Örnekler

Örnek merdiveninin on üç basamağının onu Bölüm 3–4'te işlendi; bu bölüm kalan üç basamağı çözer: özyinelemesiz birleştirmeli sıralama, melez sıralama ve büyük deney programı (Tablo 6.7). Bütün örneklerin C++ ve Python karşılıkları Ek A'dadır; her örneğin çıktısı üç dilde bayt bayt aynıdır (`esdeger_test.py`). Dosya dizini `kodlar/KOD_DIZINI.md`'dedir.

Tablo 6.7
Örnek merdiveni ve ders notundaki yerleri (çıktı her örnekte üç dilde aynı: =)

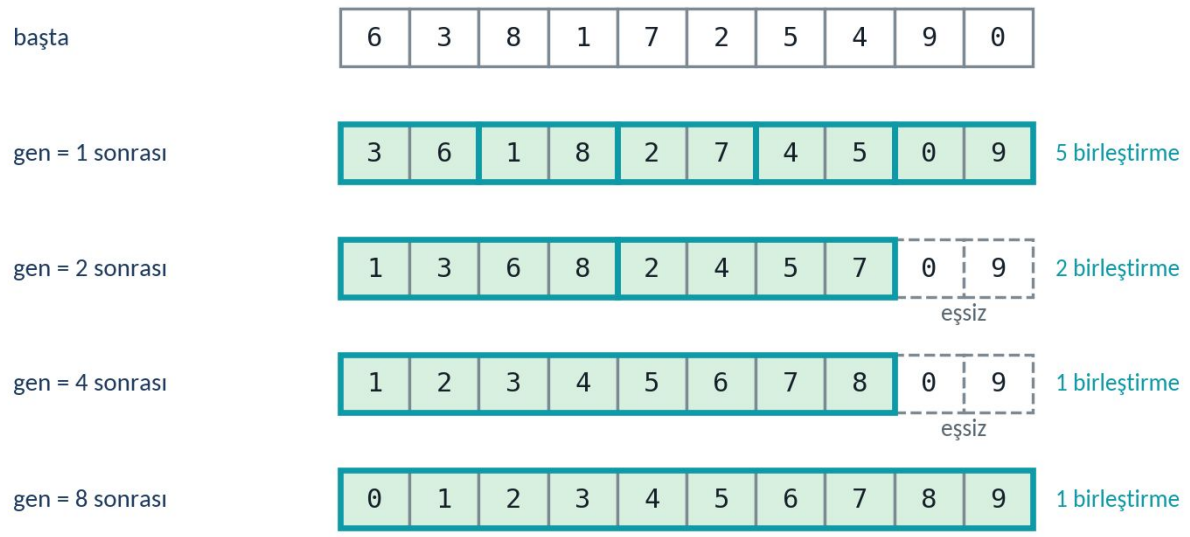
#	Örnek (kodlar/c, cpp, python)	Yeni öge	Düzey	Bölüm
1	H06_01_birlestirme	Birleştirme değişmezi	★	4.1
2	H06_02_birlestirmeli_siralama	Böl – fethet – birleştir	★★	4.1
3	H06_03_ozyineleme_agaci	Yineleme bağıntısının görselleştirilmesi	★★	3.3–3.4
4	H06_04_lomuto_hizli	Bölümleme değişmezi	★★	4.2
5	H06_05_hoare_bolumleme	Bölümleme türleri	★★	4.2
6	H06_06_pivot_deneyi	Pivot seçimi	★★	4.2, 6
7	H06_07_uc_yollu_bolumleme	Eşit anahtar sorunu	★★★	4.2
8	H06_08_ters_cevrim	Birleştirme adımını genişletme	★★	4.3
9	H06_09_quickselect	Seçim problemi	★★★	4.4
10	H06_10_asagidan_yukari	Özyinelemesiz böl ve fethet	★★★	5.1

#	Örnek (kodlar/c, cpp, python)	Yeni öge	Düzey	Bölüm
11	H06_11_melez_siralama	Sabit çarpan mühendisliği	★★★	5.2, 6
12	H06_12_buyuk_deney	Kuram–deney karşılaştırması	★★★	5.3, 6
13	H06_13_en_buyuk_alt_dizi	Aynı problemde iki strateji	★★★	4.5

5.1 Aşağıdan yukarıya birleştirmeli sıralama (Örnek 10)

Yukarıdan aşağıya sürümün özyineleme ağacı yalnızca **hangi blokların hangi sırayla** birleştirileceğini belirler. Aynı blokları özyinelemesiz de birleştirebiliriz: önce genişliği 1 olan komşu blok çiftleri, sonra 2, 4, 8, … Her geçiş diziyi bir kez tarar; ⌈lg n⌉ geçiş yeter. n ikinin kuvveti değilse son blok o geçişte eşsiz kalabilir; olduğu gibi bırakılır ve bir sonraki geçişte birleşir (Şekil 6.12).

Şekil 6.12
Aşağıdan yukarıya birleştirmeli sıralama (Örnek 10, n = 10): turkuaz çerçeve o geçişte birleştirilen blok; [8..9] iki geçiş boyunca eşsiz kalır



turkuaz çerçeve: o geçişte birleştirilen blok · 4 geçiş = ⌈lg 10⌉, 9 birleştirme, 27 karşılaştırma (Örnek 10)

c/H06_10_asagidan_yukari.c · satır 52–81

```c
int asagidan_yukari_sirala(int *a, int n, int yaz)
{
    int *yrd = malloc(((size_t)n + 1) * sizeof *yrd);
    int gecis = 0;
    if (yrd == NULL) {
        fprintf(stderr, "bellek yetersiz\n");
        exit(1);
    }
    for (int gen = 1; gen < n; gen *= 2) {
        int onceki = birlestirme_sayisi;
        for (int lo = 0; lo < n - gen; lo += 2 * gen) {
            int mid = lo + gen - 1;
            int hi = lo + 2 * gen - 1 < n - 1 ? lo + 2 * gen - 1 : n - 1;
            birlestir(a, yrd, lo, mid, hi);
        }
```

```
        gecis++;
        if (yaz) {
            int blok = (n + gen - 1) / gen;
            printf("gen = %2d :", gen);
            dizi_yaz(a, n);
            printf("   (%d birleştirme", birlestirme_sayisi - onceki);
            if (blok % 2 == 1) {
                printf("; [%d..%d] eşsiz", (blok - 1) * gen, n - 1);
            }
            printf(")\n");
        }
    }
    free(yrd);
    return gecis;
}
```

İç döngünün koşulu `lo < n - gen` sağ bloğu olmayan son bloğu atlar; `hi` dizinin sonunu aşmayacak biçimde kırpılır. Program yukarıdan aşağıya sürümle karşılaştırma sayısını da ölçer: n = 2^k olduğunda iki sürüm aynı blokları birleştirir ve sayılar eşittir; öteki n'lerde bölünme sınırları farklıdır.

Özyinelemesiz sürümün iki üstünlüğü vardır: yığın kullanmaz ve bağlı listelerde ya da dış bellekte (sıralı koşuları geçiş geçiş birleştirerek) doğal biçimde çalışır. Timsort da aşağıdan yukarıya birleştirir; ama 1, 2, 4 genişlikli bloklar yerine dizide **hazır bulunan** sıralı koşuları (run) kullanır.

5.2 Melez sıralama: küçük parçalarda araya ekleme (Örnek 11)

Hızlı sıralamanın her çağrısı bir bölümleme, bir pivot seçimi ve bir çerçeve demektir; birkaç elemanlık parçada bu yük, H5'in araya ekleme sıralamasının $\Theta(k^2)$ işinden pahalıdır. **Melez** (hybrid) sıralama, parça bir **eşik** değerinden kısa olunca bölmeyi bırakır ve parçayı araya ekleme ile bitirir. Örnek 11 üçün ortancası pivotlu, küçük parçaya özyinelemeli sürümü kullanır:

c/H06_11_melez_siralama.c · satır 86–101

```
86  void melez_sirala(int *a, int lo, int hi, int esik)
87  {
88      while (hi - lo + 1 > esik) {
89          int p = bolumle(a, lo, hi);
90          if (p - lo < hi - p) {
91              melez_sirala(a, lo, p - 1, esik);
92              lo = p + 1;
93          } else {
94              melez_sirala(a, p + 1, hi, esik);
95              hi = p - 1;
96          }
97      }
98      if (lo < hi) {
99          araya_ekle(a, lo, hi);  /* kısa parça */
100     }
101 }
```

Eşik büyüdükçe karşılaştırma ve taşıma sayısı önce biraz azalır, sonra artar (n = 10 000: eşik 1'de 146 320, 8'de 139 524, 64'te 206 532 karşılaştırma); ama bölümleme ve çağrı yükü azalır. Sayaçlar eşiğin en iyi değerini söyleyemez; bunu ancak süre ölçümü söyler (Bölüm 6.3). İş sayısı ile süre

arasındaki bu ayrım **sabit çarpan mühendisliğinin** özüdür: aynı Θ(n log n) içinde kazanç, makineye ve gerçekleştirime bağlıdır.

ÇIKTI · ÖRNEK 11

```
$ ./H06_11_melez_siralama   (sayaç tablosu, seçili eşikler; süreler stderr'de)
Melez sıralama: hızlı + araya ekleme, n = 10000 (tohum 42)
 eşik  karşılaştırma     taşıma  araya ekleme  doğru
    1         146320      73960             0  evet
    4         140007      69214          2295  evet
    8         139524      70382          1621  evet
   16         144088      77703           953  evet
   32         163572     100994           498  evet
   64         206532     148782           259  evet
```

5.3 Büyük deney programı (Örnek 12)

Örnek 12 Haftanın Deneyi'nin ölçüm programıdır: birleştirmeli, üç pivotlu hızlı sıralama (Örnek 6'nın fonksiyonları), kütüphane sıralaması ve araya ekleme aynı dosyadadır ve aynı girdi üreticisini kullanır. Argümansız çalıştırıldığında küçük bir doğrulama tablosu yazar; `ALGORITMA GIRDI N TEKRAR` argümanlarıyla tek bir yapılandırmayı ölçer ve bir CSV satırı üretir (`kodlar/deney/olcum_calistir.py` bu kipi kullanır). Rastgele pivot dizisi her sıralamada aynı tohumla (2026) başlar (`sirala` fonksiyonu, Ek A.12); böylece üç dilin karşılaştırma sayıları aynıdır:

ÇIKTI · ÖRNEK 12

```
$ ./H06_12_buyuk_deney   (seçili satırlar)
Büyük deney programı: varsayılan küçük çalıştırma (süreler stderr'de)
algoritma       girdi         n  karşılaştırma  ağaç  yığın  doğru
birlestirmeli   rastgele   1000          8731    11     11  evet
hizli_ilk       rastgele   1000         10757    21      7  evet
hizli_rastgele  rastgele   1000         11433    22      7  evet
hizli_ortanca3  rastgele   1000         10759    18      8  evet
kutuphane       rastgele   1000             -     -      -  evet
araya_ekleme    rastgele   1000        250306     0      0  evet
hizli_ilk       sirali     1000        499500  1000      2  evet
Ölçüm kipi (deney/olcum_calistir.py kullanır):
  ALGORITMA GIRDI N TEKRAR → algoritma;girdi;n;tekrar;ortanca_ms;
  en_az_ms;en_cok_ms;islemci_ms;karsilastirma;agac;yigin;dogru
```

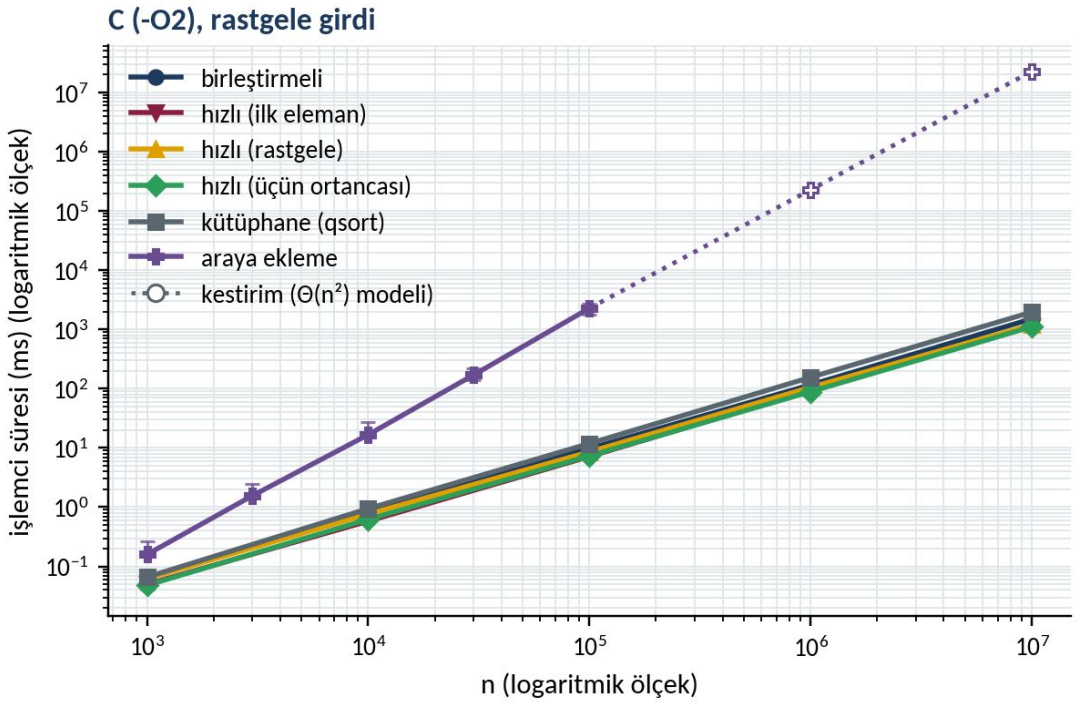
6. Haftanın Deneyi: n log n ile n^2 Karşı Karşıya

Soru. Θ(n log n) ve Θ(n^2) algoritmaları gerçek bir makinede nasıl ayrışır; aynı sınıftaki algoritmalar arasında sabit çarpanlar ne kadar fark yaratır; kötü pivot ölçümde nasıl görünür? **Düzen.** Örnek 12'nin programı birleştirmeli, hızlı (ilk / rastgele / üçün ortancası) ve kütüphane sıralamasını n = 10^3 … 10^7 rastgele (ortak LCG, tohum 42) ve sıralı girdide; araya eklemeyi C/C++'ta n ≤ 10^5, Python'da n ≤ 10^4; sıralı girdide ilk eleman pivotunu n = 2 500 … 40 000 için ölçer. Her ölçüm 1 ısınma + 5 tekrar (büyük n'de daha az), ortanca; tablolar üç bağımsız turun (Θ(n^2) yapılandırmalarında beş turun) ortancasıdır (Sedgewick & Wayne, 2011). Ortam: Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdekli paylaşımlı sanal makine; gcc 13.3.0, Python 3.11.15. Süreler **işlemci süresidir** (CPU time): makine başka işlerle paylaşıldı, duvar saati turlar arasında iki katına dek oynadı. Kendi makinenizde `kodlar/deney/` altında `python3 olcum_calistir.py && python3 kestirim.py` ile tekrarlayın: sayaçlar aynı, süreler farklı çıkacaktır.

Tablo 6.8
*C -O2, rastgele girdi: turların ortanca işlemci süresi (h06_deney_ozet.csv); * işaretli değerler ölçüm değil kestirimdir (h06_deney_kestirim.csv) ve alt sınır olarak okunmalıdır*

Algoritma (C, -O2)	n = 10^3	n = 10^4	n = 10^5	n = 10^6	n = 10^7
birleştirmeli	65,0 µs	0,891 ms	9,78 ms	116,2 ms	1,51 s
hızlı · ilk	50,0 µs	0,580 ms	7,16 ms	93,6 ms	1,13 s
hızlı · rastgele	62,0 µs	0,765 ms	8,57 ms	105,3 ms	1,20 s
hızlı · ortanca3	48,0 µs	0,624 ms	7,34 ms	89,0 ms	1,11 s
qsort	66,0 µs	0,945 ms	11,9 ms	155,6 ms	1,96 s
araya ekleme	0,159 ms	16,7 ms	2,28 s	≈ 3,8 dk*	≈ 6,3 sa*

Şekil 6.13
C -O2, rastgele girdi (log–log): beş Θ(n log n) eğrisi birbirine paralel ve eğimi ≈ 1; araya eklemenin eğimi ≈ 2, içi boş noktalar kestirim

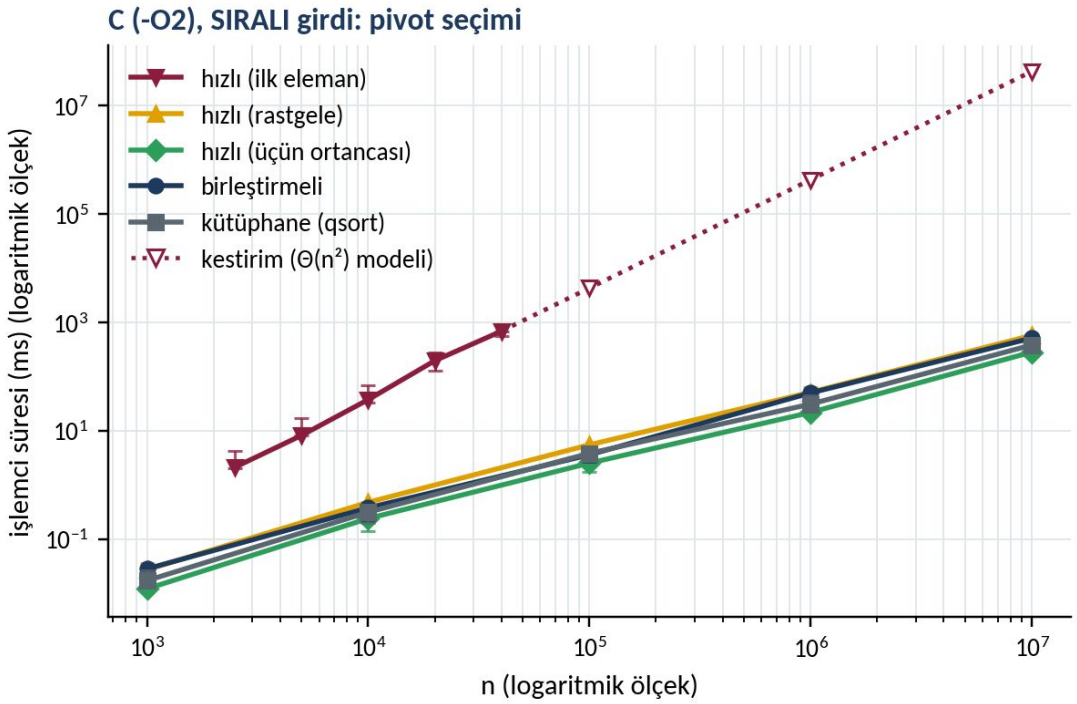


Θ(n log n) doğrulaması. n on kat büyüdükçe süre n log n için 10·lg(10n)/lg n kat büyümelidir ($10^6 \to 10^7$: 11,67); ölçülen oranlar 11,40–13,00, log–log eğimleri 1,06–1,11 (kuram 1,07–1,12); araya eklemenin eğimi 1,97–2,15 (n^2: 2). Süre/(n lg n) n = 10^7'de 4,77–8,45 ns'dir ve yataya yakındır; yalnız qsort'unki 10^7'ye doğru belirgin artar. **Aynı sınıf, farklı sabitler.** n = 10^7'de birleştirmeli sıralama, üçün ortancası pivotlu hızlı sıralamanın 1,36 katı sürdü: hızlı sıralama daha çok karşılaştırma yapar ama yerinde ve ardışık çalışır (Bölüm 3.6). C `qsort` (1,96 s) her karşılaştırmada işlev işaretçisiyle çağrı yapar; C++ `std::sort` (993,5 ms) karşılaştırmayı şablonla satır içine alır ve bizim C++ sürümümüzden (1,08 s) de hızlıdır; -O0 ile C, n = 10^6'da birleştirmelide 2,24 kat yavaşlar. **Dil sabiti ile algoritma.** Python'daki birleştirmeli sıralamamız n = 10^6'da 8,35 s (C'nin ≈ 72 katı), C ile yazılmış yerleşik `list.sort` 583,0 ms sürdü: Python'da sıralama gerekiyorsa yerleşik sıralama kullanılır; kendi sıralamamızı yazmak algoritmayı öğrenmek ve kâhinle sınamak içindir.

6.1 Karesel çöküş: sıralı girdi ve ilk eleman pivotu

Sıralı girdide ilk eleman pivotu her bölümlemede n − 1 | 0 böler (Şekil 6.8b). C -O2: n = 2 500: 2,11 ms; n = 5 000: 8,15 ms; n = 10 000: 37,2 ms; n = 20 000: 195,0 ms; n = 40 000: 683,3 ms. n iki katına çıkınca süre ≈ 4 katına çıkar (turların en az değerleriyle oranlar 3,95–4,28; ortancalar gürültülüdür). Kestirim (n^2 modeli): n = 10^5 ≈ 4,3 s*, 10^6 ≈ 7,1 dk*, 10^7 ≈ 12 sa*. Aynı sıralı 10^7 elemanı üçün ortancası 280,6 ms, rastgele pivot 572,3 ms içinde sıralar: üçün ortancası sıralı girdide her seferinde tam ortadan böler.

Şekil 6.14
Sıralı girdi (C -O2, log–log): ilk eleman pivotu eğimi ≈ 2 olan bir doğru; kesikli çizgi Θ(n^2) kestirimi; öteki stratejiler ve birleştirmeli Θ(n log n)



Derinlik (sayaçlardan, makineden bağımsız): sıralı girdide ilk eleman pivotu için ağaç yüksekliği n'dir (n = 40 000: 40 000), küçük parçaya özyineleme sayesinde gerçek yığın derinliği ise 2. Rastgele girdide n = 10^7'de ağaç yüksekliği ilk eleman pivotuyla 56, üçün ortancasıyla 47 (lg n ≈ 23,25); yığın 16 ve 17; birleştirmelide ikisi de ⌈lg n⌉ + 1 = 25.

6.2 Kestirim yöntemi ve sınanması

Ölçülemeyen boyutlar için **oran testine dayalı kestirim** kullanıldı: $t(n) = t(n_o) \cdot f(n)/f(n_o)$, n_o en büyük ölçülen boyut, f = n lg n ya da n^2. Yöntem 28 noktada sınandı (bir alt boyuttan kestirip gerçek ölçümle karşılaştırarak; h06_deney_kestirim.csv, `tur = dogrulama`): hataların ortancası %9,5; kestirim, 28 sınamanın 22 tanesinde ölçülenden **küçük** çıktı. Büyük n'de veri önbelleğe sığmaz ve işlem başına süre artar; model bunu içermez. Bu yüzden bütün kestirimler (*) **alt sınır** olarak okunmalıdır: araya ekleme 10^7 elemanı sıralamak için en az ≈ 6,3 sa ister.

6.3 Melez sıralamada eşik

Örnek 11'in eşik deneyi C ve C++'ta n = 10^6, Python'da n = 20 000 rastgele girdide eşik = 1, 4, 8, … 64 için işlemci süresini ölçer (altı ölçümün ortancası; eşik 1 = saf hızlı sıralama = 1,000). En kısa süre C'de eşik 24'te (0,889), C++'ta 16'da (0,889), Python'da 12'de (0,816) ölçüldü; ama komşu eşikler arasında bile sonuç oynuyor (Python'da 16: 1,011). Kazanç **küçüktür** ve gürültüyle aynı büyüklüktedir; en iyi eşik **bu makinede ve bu gerçekleştirimde** geçerlidir, başka derleyici, işlemci ya da veri türüyle değişir.

Kütüphaneler eşiği bu yüzden deneyle seçer (libstdc++'ta 16). **Sınırlılıklar:** tek paylaşımlı sanal makine; işlemci süresi de turlar arasında oynadı; sayaç artırımları süreye dahildir; kestirimler modeldir ve alt sınırdır. Sonuçlar büyüklük sırası ve eğim fikri verir; mutlak süreler başka makinede farklıdır.

7. Hata Avcısı

Sekiz hatanın her biri `kodlar/hata_avcisi/` altında kasıtlı hatalı ve düzeltilmiş dosya çifti olarak bulunur; belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`). Hiçbiri derleyici uyarısı üretmez; ikisi (HA2, HA7) sonucu bile bozmaz ve ancak bir araç ya da kâhinle görülür.

Tablo 6.9
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Bir dizi bitince ötekinin kalanını kopyalamamak	c: 1 2 5 5 6 7 9 0 0; yazılmamış (0 kalan) hücre: 2	Ana döngü i < na ve j < nb iken çalışır; kalanlar için döngü yok	İki "kalan" döngüsü
2	Her çağrıda tampon ayırıp bırakmamak	malloc çağrısı = 999 · ayrılan = 39904 bayt · free = 0; LeakSanitizer: 39904 byte(s) leaked in 999 allocation(s).	malloc her birleştirmede, free yok; sıralama doğru olduğundan görünmez	Tamponu bir kez ayır, paylaş, sonda bırak
3	Python'da dilimle özyinelemek (`a[:p]`)	sonra: [3, 1, 2, 4, 7, 8, 5, 6]; sıralı değil	Dilim yeni bir listedir: özyineleme kopyayı sıralar	Aynı liste + (lo, hi) indisleri
4	Orta nokta hatası: `(lo + hi + 1) / 2`	[0..1] aralığı küçülmez: C'de SIGSEGV (139), Python'da RecursionError	İki elemanlı aralıkta mid = hi: sol parça aralığın kendisi	mid = lo + (hi − lo)/2
5	Hoare'un dönüşünü pivotun yeri sanmak	sonuç: 1 3 4 5 2 6 7 8; 1000 rastgele dizinin (n = 20) 999 tanesi sıralı DEĞİL	j yalnızca sınırdır; [lo..j−1] a[j]'yi dışarıda bırakır	[lo..j] ve [j+1..hi]
6	Birleştirmede eşitlikte sağdan almak (`<`)	60 Ufuk, 60 Ece, 70 Selin, 70 Deniz, 70 Ali, 85 Naz, 85 Can	Eşit anahtarda sağdaki önce alınır	`<=`: eşitlikte soldan
7	Hızlı sıralamayı kararlı sanmak	60 Ufuk, 60 Ece, 70 Ali, 70 Deniz, 70 Selin, 85 Can, 85 Naz	Bölümlemedeki uzak takaslar eşitlerin sırasını bozar	Kararlı algoritma ya da (puan, ad) anahtarı
8	Sıralı girdide ilk eleman pivotu	sıralı n = 4000: karşılaştırma = 7998000 (n(n-1)/2 = 7998000) · derinlik = 4000; yığın 256 KB'ta ters n = 20 000: SIGSEGV; Python n = 2000: RecursionError	Her bölünme n − 1	0: n(n − 1)/2 karşılaştırma, n derinlik	Rastgele pivot / üçün ortancası + küçük parçaya özyineleme

HA1–HA2 · Kalanı kopyalamamak; her çağrıda tampon.

HA1'de ana döngü bir dizi bitince durur ve ötekinin kalanı (b'nin 10 ve 12'si) hiç yazılmaz; belirti girdiye bağlıdır, bu yüzden testte **iki yön de** denenmelidir (Örnek 1'in tablosu na = 0 ve nb = 0 durumlarını içerir). HA2'de `bs_ozyineli` her birleştirmeden önce `int *yrd = malloc(boy * sizeof *yrd);` ile tampon ayırır ve hiç bırakmaz: sıralama doğrudur, olağan çalıştırmada belirti yoktur; yalnızca sayaçlar ve LeakSanitizer (`-fsanitize=address`) sızıntıyı gösterir. `free` eklemek sızıntıyı kapatır ama gereksiz 999 ayırmayı bırakır; doğru çözüm tek tampondur (Bölüm 3.6).

HA3 · Dilim bir kopyadır (Python).

Python `hata_avcisi/H06_HA3_dilim_kopyasi.py · satır 30–31`

```
30      hizli_sirala(a[:p])          # HATA: kopya sıralanır, a değil
31      hizli_sirala(a[p + 1:])      # HATA: kopya sıralanır, a değil
```

Yalnızca en üstteki bölümleme asıl listeye yansır: [3, 1, 2] | 4 | [7, 8, 5, 6]; parçalar kopyalarda sıralanır ve atılır. Birleştirmeli sıralamada dilimli sürüm (sonucu döndürdüğü için) doğrudur ama her çağrıda yeni listeler ayırır (Bölüm 8, Alıştırma 6.15).

HA4 · Orta nokta ve sonsuz özyineleme.

C `hata_avcisi/H06_HA4_orta_indis.c · satır 47–49`

```
47      int mid = (lo + hi + 1) / 2;        /* HATA: yukarı yuvarlama */
48      bs_ozyineli(a, yrd, lo, mid, d + 1);
49      bs_ozyineli(a, yrd, mid + 1, hi, d + 1);
```

[0..1] aralığında mid = 1 = hi olur: sol çağrı yine [0..1]'dir. Program varsayılan çalıştırmada derinliği 10 000'de keser; `korumasiz` argümanıyla SIGSEGV ile çöker (çıkış kodu 139), Python sürümü RecursionError verir. Aşağı yuvarlayan `lo + (hi − lo)/2` iki parçanın da küçüldüğünü güvenceye alır ve `(lo + hi)/2`'nin aksine büyük indislerde taşmaz.

HA5 · Hoare'un j'si.

C++ `hata_avcisi/H06_HA5_hoare_donus.cpp · satır 41–42`

```
41      hizli_hoare(a, lo, p - 1);          // HATA: a[p] dışarıda kalıyor
42      hizli_hoare(a, p + 1, hi);
```

Lomuto alışkanlığıyla [lo..p − 1] ve [p + 1..hi] yazılırsa a[p] hiçbir parçaya girmez; bu eleman pivot olmadığı için yanlış yerde kalabilir. Program 1000 rastgele dizinin 999'unda yanlış sonuç bulur; düzeltilmiş sürüm 0.

HA6–HA7 · Kararlılık.

İki hata aynı belirtiyi farklı nedenle verir. HA6'da birleştirme `a[i][0] < a[j][0]` yazar: eşitlikte sağdaki önce alınır ve sıralama kararsızlaşır (`kararlı mı: HAYIR`). HA7'de algoritmanın kendisi kararsızdır: Lomuto'nun uzak takasları eşit puanlıların ad sırasını bozar; çözüm kararlı bir algoritma ya da bileşik anahtardır: `hızlı + (puan, ad)     : 60 Ece, 60 Ufuk, 70 Ali, 70 Deniz, 70 Selin, 85 Can, 85 Naz`.

HA8 · Sıralı girdi ve ilk eleman pivotu.

C `hata_avcisi/H06_HA8_sirali_girdi.c · satır 28–31`

```
28  static int bolumle(int *a, int lo, int hi)
29  {
30      takas(a, lo, hi);                   /* HATA: pivot hep ilk eleman */
31      int pivot = a[hi];
```

Sıralı girdide karşılaştırma sayısı tam n(n − 1)/2'dir ve derinlik n'ye eşittir. Varsayılan 8 MB yığında bu makinede taşma görülmediği için yığın 256 KB'a küçültüldü: ters girdide n = 20 000 çöker. Sıralı girdide -O2 ile çökmez: büyük parça son (kuyruk) çağrıdadır ve GCC onu döngüye çevirir (Bölüm 3.6). Düzeltilmiş sürüm (rastgele pivot + küçük parçaya özyineleme) aynı küçük yığında n = 20 000 ters girdiyi 10 derinlikle sıralar; Python sürümü n = 2000'de RecursionError verir.

8. Dil Penceresi: C · C++ · Python

Tablo 6.10
Bu haftanın dil farkları (rastgele pivot için üç dilde de ortak LCG kullanıldı)

Öğe	C	C++	Python
Kütüphane sıralaması	qsort: algoritma ve kararlılık belirtilmez	std::sort (introsort, kararsız), std::stable_sort	sorted / list.sort (Timsort, kararlı)
Birleştirme	elle	std::merge, std::inplace_merge	heapq.merge
k'ıncı en küçük	elle (quickselect)	std::nth_element	heapq.nsmallest; statistics.median (sıralar)
Alt dizi	işaretçi + uzunluk: kopyasız	yineleyici aralığı: kopyasız	a[lo:hi] yeni liste: O(k) zaman ve bellek
Tampon	malloc / free	std::vector (RAII)	liste (çöp toplayıcı)
Takas	geçici değişken	std::swap	a[i], a[j] = a[j], a[i]
İki değer döndürmek	çıkış parametresi / struct	std::pair + auto [lt, gt]	demet: lt, gt = ...
do ... while	var	var	yok: önce adım, sonra while
Büyük sayaç	long long (int taşar)	long long	int (sınırsız)
Derinlik sınırı	yığın (ulimit -s): SIGSEGV	C ile aynı	sys.getrecursionlimit() = 1000: RecursionError
Rastgele pivot	rand() (dağılım belirsiz)	std::mt19937	random.randint

Kütüphane karşılıkları (Kavram 6). Aynı beş kaydı (puan, ad) her dil kendi kütüphanesiyle sıralar; C++ ve Python ayrıca iki sıralı diziyi birleştirir ve Örnek 9'un notlarının ortancasını bulur. Python `statistics.median` ortancayı sıralayarak bulur; C++ `std::nth_element` ise quickselect ailesindendir (libstdc++'ta introselect):

ÇIKTI · KAVRAM 6

```
$ ./H06_K06_kutuphane   (C++)
stable_sort: 60 Ece  60 Ufuk  70 Ali  70 Can  85 Naz
merge: 1 2 5 5 6 7 9 10 12
nth_element ortanca: 72
sort: 1 2 3 4 5 6 7 8
```

C `qsort` çıktısı (`qsort: 60 Ece  60 Ufuk  70 Ali  70 Can  85 Naz`) bu makinede kararlıdır: Kavram 3'ün gözlemine göre bu glibc'de qsort birleştirmeli sıralama gibi davranır (Bölüm 3.6), ama standart bunu güvence etmez. **C++ std::sort ise kararsızdır** (introsort: bölümlemenin uzak takasları eşitlerin sırasını bozar, bkz. Hata Avcısı 7); küçük dizilerde yalnızca araya ekleme kullandığından kararlı **görünebilir**. Kararlılık küçük testlerle doğrulanamaz, belgelenmiş bir güvencedir (std::stable_sort); Alıştırma 6.16'da bunu kendiniz ölçeceksiniz.

Dilimleme maliyeti (Kavram 5). Python'da `a[lo:hi]` yeni bir liste oluşturur ve k elemanı kopyalar: O(k) zaman ve bellek. Dilim değiştirildiğinde asıl liste değişmez (HA3'ün nedeni); atama ise kopya değil başvurudur:

ÇIKTI · KAVRAM 5

```
$ python3 H06_K05_dilim.py   (b = a[2:5]; b[0] = 99; c = a; c[0] = 0)
a = [6, 3, 8, 1, 7, 2, 5, 4]
b = [99, 1, 7] · b is a: False
c = a; c[0] = 0 → a = [0, 3, 8, 1, 7, 2, 5, 4]
özyineleme sınırı: 1000
```

Dilimli birleştirmeli sıralama doğrudur ve kopyalanan eleman sayısı indisli sürümle aynı düzendedir; fark her çağrıda yeni liste ayırmasıdır. Bu makinede iki sürümün süresi $n = 2 \cdot 10^5$'e dek yakın çıktı: dilimin kendisi C'de yazılmış hızlı bir kopyadır. Dolayısıyla "dilim her zaman yavaşlatır" denemez; doğru ifade "dilim ek bellek ayırır ve kopyalar"dır. Hızlı sıralamada ise dilim **yanlıştır**, çünkü algoritma yerindedir. **Özyineleme derinliği:** kötü pivotla Python'da RecursionError, C ve C++'ta yığın taşması (HA8); bu hafta sınır yükseltilmez, derinlik algoritmayla (küçük parçaya özyineleme) sınırlanır.

9. Mühendis Gözüyle

En kötü durum bir güvenlik sorunudur. Ortalama durumu iyi olan bir algoritma, girdiyi seçebilen biri için en kötü durumuyla çalışır. McIlroy (1999), karşılaştırma fonksiyonuna yanıt verirken girdiyi "o anda" kuran bir **katil rakip** (killer adversary) ile pek çok `qsort` gerçekleştirimini karesel davranışa zorladı: rakip, pivot olarak seçilecek elemanı tahmin edip ona hep en küçük değeri verir. Crosby ve Wallach (2003) aynı fikrin ağ hizmetlerine karşı nasıl kullanılabileceğini gösterdi: saldırgan, sunucunun algoritmasını en kötü durumunda çalıştıran girdiler göndererek az veriyle çok işlemci zamanı tüketir (algorithmic complexity attack).

Kavram 4 (Atölye 2'nin aracı; tam tablo Bölüm 11.9'da) bir girdinin üç pivot stratejisini ne kadar zorladığını oran = karşılaştırma / $(n(n-1)/2)$ ile ölçer (1,00 tam karesel). $n = 1000$'de **organ borusu** (önce artan, sonra azalan) deseni ilk eleman ve üçün ortancasını 0,17 / 0,17 oranına çıkarır; rastgele pivot 0,02 oranında kalır. **Tümü eşit** girdi üç stratejiyi birden çökertir (1,00, 1,00, 1,01): sorun pivot seçiminde değil, Lomuto'nun eşitleri tek tarafa yığmasındadır; çözüm üç yollu bölümleme ya da Hoare'dur (Örnek 5, 7).

Mühendisin üç savunması. (1) **Rastgele pivot:** hiçbir sabit girdi en kötü durumu tetikleyemez; ama rastgele sayı üretecinin tohumu tahmin edilebiliyorsa savunma zayıflar. (2) **Introsort** (Musser, 1997): özyineleme derinliği $2 \cdot \lfloor \lg n \rfloor$'yi aşınca öbek sıralamasına geç; en kötü durum $O(n \log n)$ olur ve ortalama hız korunur. libstdc++ `std::sort` budur. (3) **Garantili algoritma:** en kötü durumu $\Theta(n \log n)$ olan birleştirmeli sıralama (Python, Java nesne dizileri); bedeli $O(n)$ ek bellektir. Kural: **kullanıcıdan gelen veriyi sıralıyorsanız en kötü durumu sınırlı bir algoritma kullanın** ve "benim verim rastgele" varsayımına güvenmeyin.

> MÜHENDİS GÖZÜYLE · **Sayaç, kâhin ve ölçüm: üç ayrı tanık**
>
> Bu haftanın iddiaları önce **sayaçla** doğrulandı (makineden bağımsız; kuramla birebir: $2n - 1$ çağrı, $n(n-1)/2$, $2(n+1)H(n) - 4n$); her hızlı algoritmanın yanında yavaş ama apaçık doğru bir **kâhin** var (kütüphane sıralaması, kaba çift taraması, sıralayıp a[k]'ya bakmak, Kadane) ve silinmez, bir sonraki iyileştirmenin testi olur; **süre** en son ölçüldü ve ortamıyla birlikte verildi: eşik deneyindeki gibi küçük etkiler ancak tekrarlı ölçümle ve "bu makinede" kaydıyla söylenebilir.

10. Mini-OBS 2 v2.5: Büyük Veride Böl ve Fethet

Bu hafta: Mini-OBS'nin büyük veri sorunlarını böl ve fethetle çözüyoruz. Birleştirmeli sıralamayı (tek tampon, kararlı) ve Hoare bölümlemeli hızlı sıralamayı v2.4'ün qsort arayüzünde yazıyoruz; yükleyiciyi

"oku → anahtarları sırala → komşuları karşılaştır → bir kez taşı" düzenine çeviriyoruz; yinelenen numara tespitini sıralamayla O(n log n)'e indiriyoruz; iki şubenin listelerini birleştirme adımıyla birleştiriyoruz; ortancayı quickselect ile buluyoruz. Her yeni algoritma bir önceki sürümün yöntemine (kâhin) karşı sınanır.

Mini-OBS 2 v2.5'in hızlı sıralaması **Hoare** bölümlemesini kullanır (eşit numaralarda dengeli böldüğü için); bu haftanın örnek merdiveninin ana sürümü ise izlemesi ve değişmezi daha yalın olan **Lomuto**'dur. Sarmal, önceki sürümlerin ölçümleriyle kapanır: 10^6 kayıtlık yükleme v1.0'da ≈ 1,6 saat (kestirim) iken v2.5'te 2,28 s sürer (≈ 2 540 kat); 10^5 kayıtta yinelenen numara tespiti v2.1'in tüm çiftler taramasında 4,33 s, sıralayarak 23,1 ms (187 kat; sonuç aynı); 10^6 kayıtlık iki şubeyi birleştirmek 134,0 ms, uç uca ekleyip sıralamak 918,3 ms; 10^6 ortalamanın ortancası quickselect ile 22,0 ms, sıralamayla 208,2 ms (proje belgesinin ölçümleri).

Gereksinimler

- `bolfethet.h/.c`: `birlestir_genel` (kararlı; değişmez yorumu), `birlestirmeli_sirala_say` (tek tampon, bir kez ayrılır), `hizli_sirala_say` (Hoare bölümlemesi, üçün ortancası, küçük parçaya özyineleme; isteğe bağlı derinlik çıktısı), `k_inci_kucuk`, `ortanca_quickselect`; arayüz ve sayaçlar v2.4 ile aynı.
- Yükleyici (`sinif_yukle`) toplu kipte: sonuç numara sırasında; mükerrer ya da bozuk satırda v1.0 ile **aynı durum ve satır**; Θ(n log n).
- `yinelenen_bul_siralama`: v2.1 ile aynı çiftler, aynı sözlük sırası; `denetle --yontem siralama|ciftler`.
- `sube_birlestir` + `birlestir` programı: numara sırasında birleşik liste, çakışmaların raporu, çıkış kodu 1.
- `ortanca` programı: üç dersin ortancası; `--denetle` ile sıralama kâhini.
- `listele`'ye `birlestirmeli` (varsayılan) ve `hizli`.
- Testler: elle izlenmiş sayaçlar; kâhinler (qsort, v2.1, v1.0 + qsort, uç uca ekleme + sıralama); kararlılık.
- Ölçüm raporu: yükleme (v1.0, v2.3, v2.5; 10^3–10^6), yinelenen tespiti (v2.1 ↔ v2.5; aynı sonuç ve hızlanma), birleştirmeli ↔ hızlı ↔ qsort (en az 10^4–10^6, girdi sırası), şube birleştirme ↔ uç uca + sırala, quickselect ↔ sıralama; tablolar, log–log grafikler, yorum.

İpuçları

- Orta noktayı `lo + (hi − lo) / 2` ile hesaplayın (v2.3); yarı açık aralıkta [lo, orta) ve [orta, hi).
- Birleştirmede eşitlikte **soldan** alın (`b < a` ise sağdan): kararlılık bu tek karşılaştırmadadır.
- Tamponu özyinelemenin dışında bir kez ayırın; özyinelemeli yardımcı fonksiyona işaretçi olarak verin.
- Hoare bölümlemesinde pivotun **değerini** bir kenara kopyalayın: takaslar sırasında pivotun bulunduğu hücre değişebilir.
- Hoare'un döndürdüğü j ile [lo, j] ve [j + 1, hi]; Lomuto'nun döndürdüğü p ile [lo, p − 1] ve [p + 1, hi]. İkisini karıştırmayın (`--hata-goster hoare`).
- Yükleyicide satır numaralarını ayrı bir dizide tutun; anahtar (numara, indis) olsun, kayıt değil.
- Quickselect'i kopya üzerinde çalıştırın: ortanca bulmak sınıfın kayıt sırasını değiştirmemelidir.
- Kâhin uyuşmazlık bulursa önce küçük girdileri deneyin: 3–4 öğelik bir karşı örnek, 300 öğelik olandan çok daha kolay izlenir.

Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. `olcer anahtar_birlestirmeli --sira artan --n 1000` karşılaştırma 4 932, taşıma 19 952 (“kuram: UYUMLU”); `--sira azalan` karşılaştırma 5 044.
3. `olcer anahtar_hizli --sira artan --n 1024`: derinlik 11 (= $\lfloor \log_2 n \rfloor + 1$); karışık girdide derinlik bu sınırın altında.
4. İki mükerrer numaralı dosyada (proje belgesi, bölüm 5.1) `mini_obs` “satır 5” raporlar ve 1 ile çıkar — v1.0 ile aynı.
5. `denetle testler/yinelenen_k3.csv` ve `--yontem ciftler` aynı üç çifti aynı sırada yazar; ikisi de 1 ile çıkar; sıralayarak yöntemin karşılaştırması 148'i aşmaz.
6. `birlestir ornek_veri/ogrenciler.csv ornek_veri/sube_b.csv`: 14 kayıt, 10 karşılaştırma, 14 taşıma, çıkış 0; `testler/sube_cakisma.csv` ile “ÇAKIŞMA: 1 numara … 2301010047”, çıkış 1.
7. `ortanca ornek_veri/ogrenciler.csv --denetle`: ALG 77,50, MAT 69,30, FIZ 55,30; üçünde “aynı EVET”.
8. `listele testler/kararlilik.csv --yontem hepsi`: birlestirmeli “EVET / EVET”, hizli “HAYIR / HAYIR”.
9. Kendi makinenizde 10^6 kayıtlık dosya toplu kipte 1 dakikanın altında yüklenir (bu makinede 2,28 s); 10^5 kayıtta yinelenen tespiti v2.1'den en az 50 kat hızlıdır (bu makinede 187 kat).
10. `--hata-goster kuyruk` ve `--hata-goster hoare` uyuşmazlığı ve en küçük karşı örneği basar, 1 ile çıkar; `make asan` temizdir.

Tablo 6.11
Mini-OBS 2 v2.5 test senaryoları (proje belgesinden)

#	Girdi	Beklenen çıktı / davranış
1	birleştir [1, 3, 5, 7] + [2, 3, 6] (anahtar, kimlik)	1 2 3^A 3^B 5 6 7; 6 karşılaştırma, 7 taşıma
2	birleştirmeli sırala 5 2 4 6 1 3	1 2 3 4 5 6; 11 karşılaştırma, 32 taşıma
3	(2, 0) (2, 1) (1, 2): birleştirmeli ↔ hızlı	(1, 2) (2, 0) (2, 1) ↔ (1, 2) (2, 1) (2, 0)
4	hızlı sırala 1 000 eşit öğe	sıralı; derinlik ≤ 10
5	k_inci_kucuk(7 2 9 4 4 1 8, k = 3)	4; solunda ≤ 4, sağında ≥ 4
6	ortanca {4, 1, 3, 2} · {70 × 6}	2,5 · 70 (17 karşılaştırma)
7	yinelenen numaralar 7 3 7 5 3 7	(0,2) (0,5) (1,4) (2,5); 11 + 5 karşılaştırma
8	CSV: 5, 2, 9, 9, 2 (satır 1–5)	mükerrer, satır 4 (v1.0 ile aynı)
9	CSV: 5, 5, bozuk satır	mükerrer, satır 2
10	CSV: 5, 2, bozuk satır, 2	biçim hatası, satır 3
11	`birlestir` A + `testler/sube_cakisma.csv`	çakışma 2301010047; birleşik listede ÖZTÜRK (A'nın kaydı); çıkış 1
12	`test_kahin --hata-goster kuyruk` / `hoare`	en küçük karşı örnek [2, 1] → [1, 1] / [2, 3, 1] → [2, 1, 3]; çıkış 1

Öğrenciye verilen dosyalar (kendi v2.4'ünüz, `bolfethet.h`, `sube.h`, örnek veriler, ölçüm düzeneği), yazılacak modüller ve dört meydan okuma görevi (melez birleştirmeli sıralama, yarım tampon, Lomuto

ile Hoare'un çok tekrarlı girdide karşılaştırılması, doğal birleştirmeli sıralama) Mini-OBS 2 v2.5 proje belgesindedir.

11. Alıştırmalar ve Atölyeler

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons, F) kod yazma (★ / ★★ / ★★★), G) diller arası okuma. Kodu verilen her alıştırmanın dosyası `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 6.1 ★☆☆ *Çıktı tahmini*

`bs(0, 5)` çağrısının özyineleme ağacını çizin. Program hangi aralıkları hangi sırayla yazar ve `bs` toplam kaç kez çağrılır? (İpucu: yazdırma iki çağrının **ardından**, birleştirmenin yapılacağı yerdedir; 2n − 1 kuralını hatırlayın.)

alistirmalar/H06_A01_cagri_sirasi.c · satır 12–22

```c
static void bs(int lo, int hi)
{
    cagri++;
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;
    bs(lo, mid);
    bs(mid + 1, hi);
    printf("[%d..%d] ", lo, hi);  /* birleştirme burada yapılırdı */
}
```

Alıştırma 6.2 ★★☆ *İzleme*

[5, 2, 7, 1, 6, 3] dizisini önce **Lomuto** (pivot = a[hi]) ile bölümleyin: her j adımından sonra i'yi ve diziyi yazın; dönüş p, karşılaştırma ve takas sayısı nedir? Sonra özgün diziyi **Hoare** (x = a[lo]) ile bölümleyin: her turda i ile j'nin durduğu konumları ve takası yazın; dönüş j nedir, pivot 5 nerededir? İki yöntemin özyinelemeyle çağıracağı aralıkları karşılaştırın.

Alıştırma 6.3 ★★☆ *İzleme*

Örnek 7'nin üç yollu bölümlemesini [1, 2, 0, 2, 1, 0, 1] dizisine p = 1 ile uygulayın. Her adımda lt, i, gt'yi ve yapılan işlemi yazın. Döngü bittiğinde lt ve gt kaçtır, kaç takas yapılmıştır? a[i] > p dalında i'nin neden artmadığını bu izlemedeki bir adımla gösterin.

11.2 Karmaşıklık (B)

Alıştırma 6.4 ★☆☆ *Ana teorem*

Basitleştirilmiş ana teoremle çözün; her birinde a, b, d'yi ve hangi durumun geçerli olduğunu yazın: (a) $T(n) = 3T(n/2) + n$; (b) $T(n) = 4T(n/2) + n^2$; (c) $T(n) = T(n/2) + n$; (d) $T(n) = 2T(n/4) + 1$; (e) $T(n) = 8T(n/2) + n^2$; (f) $T(n) = 9T(n/3) + n$. Sonuçlarınızı web sayfasındaki ana teorem hesaplayıcısıyla doğrulayın.

Alıştırma 6.5 ★★☆ *Özyineleme ağacı*

(a) T(n) = T(n/3) + T(2n/3) + n bağıntısının özyineleme ağacının ilk üç düzeyini çizin; her düzeyin toplam işini ve en uzun yolun boyunu bulun; Θ sınıfı nedir? (b) Hızlı sıralamada her bölümleme diziyi 1/10 – 9/10 oranında bölseydi bağıntı ne olurdu, çözümü nedir? Bu sonuç "pivotun tam ortanca olması gerekmez" sözünü nasıl destekler? (c) T(n) = 2T(n/2) + n·lg n basitleştirilmiş ana teoremin kalıbına neden uymaz? Özyineleme ağacıyla çözün.

Alıştırma 6.6 ★★☆ *Karmaşıklık*

Quickselect için (a) pivot her turda aralığı tam yarıya bölüyorsa, (b) pivot her turda aralığın en küçük elemanıysa yineleme bağıntısını yazıp çözün. (c) Örnek 9'un çıktısında karşılaştırma/n oranı 4–6 arasındadır; (a)'daki sezginin verdiği 2'den neden büyüktür? İki nedeni kodla ilişkilendirerek açıklayın.

11.3 Değişmez yazma (C)

Alıştırma 6.7 ★★☆ *Değişmez yaz*

Örnek 1'deki `birlestir` fonksiyonunun (`c/H06_01_birlestirme.c · satır 24–41`) ana döngüsü için c[0..k − 1], a[0..i − 1] ve b[0..j − 1] cinsinden bir değişmez yazın. Değişmezin döngüden önce doğru olduğunu, her turda korunduğunu ve döngü + iki kalan döngüsü bittiğinde c'nin sıralı olduğunu gösterin. Kararlılık hangi karşılaştırmaya bağlıdır?

Alıştırma 6.8 ★★☆ *Değişmez yaz*

Örnek 7'nin `uc_yollu_bolumle` fonksiyonu için lt, i ve gt cinsinden dört bölgeli değişmezi yazın. Üç dalın (a[i] < p, a[i] > p, a[i] = p) her birinin değişmezi koruduğunu ve döngünün sonlandığını (hangi nicelik her turda azalır?) gösterin.

11.4 Hata bulma (D)

Alıştırma 6.9 ★★☆ *Hata bul*

[6, 3, 8, 1, 7, 2, 5, 4] için 1 2 3 4 5 6 7 8 beklenirken program `1 1 2 2 3 1 4 4` yazıyor. Hatalı satırı bulun ve tek karakterle düzeltin. Bazı değerlerin iki kez görünmesini hatayla açıklayın.

`alistirmalar/H06_A02_hata_bul_geri_kopya.c · satır 10–28`

```
10  static void birlestir(int *a, int *yrd, int lo, int mid, int hi)
11  {
12      int i = lo, j = mid + 1, k = lo;
13      while (i <= mid && j <= hi) {
14          if (a[i] <= a[j]) {
15              yrd[k++] = a[i++];
16          } else {
17              yrd[k++] = a[j++];
18          }
19      }
20      while (i <= mid) {
21          yrd[k++] = a[i++];
22      }
23      while (j <= hi) {
```

```
24          yrd[k++] = a[j++];
25      }
26      for (k = lo; k < hi; k++) {  /* geri kopyala */
27          a[k] = yrd[k];
28      }
```

Alıştırma 6.10 ★★☆ *Hata bul*

[2, 4, 1, 3, 5] dizisinde program `birleştirmeli = 1 · kaba = 3` yazıyor. Birleştirmeli sayım hangi satırda yanlış? Kök birleştirmesini elle izleyerek eksik sayılan çiftleri gösterin ve düzeltin.

alistirmalar/H06_A03_hata_bul_ters_cevrim.c · satır 9–19

```
 9  static long long birlestir_say(int *a, int *yrd, int lo, int mid, int hi)
10  {
11      long long sayi = 0;
12      int i = lo, j = mid + 1, k = lo;
13      while (i <= mid && j <= hi) {
14          if (a[i] <= a[j]) {
15              yrd[k++] = a[i++];
16          } else {
17              yrd[k++] = a[j++];
18              sayi += mid - i;  /* soldaki kalanlar a[j]'den büyük */
19          }
```

11.5 Parsons (E)

Alıştırma 6.11 ★★☆ *Parsons*

Aşağıdaki satırlar Hoare bölümlemesinin karıştırılmış satırlarıdır (dosya tamamlanacak; bu hâliyle derlenmez). İlk üç satır yerindedir. Doğru sırayı ve girintiyi yazın; [6, 3, 8, 1, 7, 2, 5, 4] için sonuç Örnek 5'teki gibi dönüş j = 4 ve 4 3 5 1 2 7 8 6 olmalıdır.

alistirmalar/H06_A04_parsons_hoare.c · satır 6–22

```
int hoare_bolumle(int *a, int lo, int hi)
{
    int x = a[lo];
    int i = lo - 1, j = hi + 1;
        if (i >= j) {
        } while (a[i] < x);
    for (;;) {
            j--;
            return j;
        takas(a, i, j);
        do {
        } while (a[j] > x);
            i++;
        do {
        }
    }
}
```

11.6 Kod yazma (F)

Alıştırma 6.12 ★☆☆ *Kod yaz*

İki **sıralı** tamsayı dizisinin (tekrar içerebilir) ortak değerlerini, her değer bir kez olmak üzere, birleştirme kalıbıyla Θ(na + nb) zamanda bulan `kesisim` fonksiyonunu C'de yazın; karşılaştırma sayısını sayın. Kâhin: iç içe iki döngü. Örnek: x = {1, 3, 3, 5, 8, 9}, y = {2, 3, 3, 4, 8, 10, 12} → 3 8. Rastgele sıralı dizilerde (ortak LCG, tohum 42) sonucu kâhinle karşılaştırın ve karşılaştırma sayısının na + nb − 1'i aşmadığını gösterin. C++ ve Python karşılıklarını da yazın.

Alıştırma 6.13 ★★☆ *Kod yaz*

Örnek 5'in Hoare bölümlemesini **rastgele pivotla** (seçilen eleman önce a[lo] ile takas edilir) ve **küçük parçaya özyineleme, büyüğe döngü** tekniğiyle (Örnek 6) birleştirin. n = 100 000 sıralı, ters, tümü eşit ve rastgele girdide karşılaştırma, takas ve yığın derinliğini yazdırın: derinlik hiçbir girdide ⌊lg n⌋ + 1'i aşmamalı, tümü eşit girdide Lomuto'nun karesel çöküşü (Örnek 5) görülmemeli.

Alıştırma 6.14 ★★★ *Kod yaz*

Karatsuba çarpımı. n = 2^k katsayılı iki polinomu çarpın: p = p_A + x^h·p_U, q = q_A + x^h·q_U (h = n/2) yazıp dört yerine **üç** alt çarpım yapın: z_0 = p_A·q_A, z_2 = p_U·q_U, z_1 = (p_A + p_U)(q_A + q_U) − z_0 − z_2. Katsayı çarpımlarını sayın ve n = 1 … 1024 için okul yönteminin n^2 çarpımıyla karşılaştırın; sonucu okul yöntemiyle (kâhin) doğrulayın. Bağıntıyı yazıp ana teoremle çözün; ölçtüğünüz çarpım sayısı neye eşittir? (Kleinberg & Tardos, 2006)

11.7 Diller arası okuma (G)

Alıştırma 6.15 ★★☆ *Diller arası okuma*

Dilimli birleştirmeli sıralama (`birlestir` iki listeden yeni bir liste döndürür). Program a'yı ve s'yi yazdırınca a değişmiş midir? `liste_sayisi` en az kaçtır (her dilim ve birleştirme yeni bir liste)? Örnek 2 (C) kaç dizi ayırır? Hata Avcısı 3'teki hızlı sıralama dilimle neden **yanlış**, bu sıralama neden **doğrudur**?

alistirmalar/H06_A05_dilimli_bs.py · satır 24–30

```
def bs(a):
    global liste_sayisi
    if len(a) <= 1:
        return a
    m = len(a) // 2
    liste_sayisi += 2                  # a[:m] ve a[m:] yeni listelerdir
    return birlestir(bs(a[:m]), bs(a[m:]))
```

Alıştırma 6.16 ★★☆ *Diller arası okuma*

Program 0, 1, 2 anahtarlı n kaydı yalnız anahtara göre `std::sort` ve `std::stable_sort` ile sıralayıp giriş sırası bozulan eşit komşuları sayar. n = 8, 16, 40, 1000 için tahmininiz nedir? n ≤ 16'da `std::sort`'un 0 vermesi kararlılık mı gösterir (Bölüm 3.6: introsort eşiği 16)? Python'da `sorted` ne gösterirdi?

alistirmalar/H06_A06_sort_kararlilik.cpp · satır 21–31

```
    auto kars = [](const auto& x, const auto& y) { return x.first < y.first; };
    if (kararli) {
```

```
        std::stable_sort(v.begin(), v.end(), kars);
    } else {
        std::sort(v.begin(), v.end(), kars);
    }
    int b = 0;
    for (int i = 1; i < n; i++) {
        b += v[i - 1].first == v[i].first && v[i - 1].second > v[i].second;
    }
    return b;
```

11.8 Atölye 1 · Kâğıt birleştirme (gruplar, ≈ 20 dk)

ALGORİTMA ATÖLYESİ · **İki sıralı desteyi birleştirin, sonra birleştirmeli sıralamayı canlandırın**

- **Birleştirme yarışı (eşli).** Her çifte 1–20 arası 16 kart verilir; kartlar iki sıralı 8'lik desteye ayrılır. Bir kişi birleştirir, öteki her "hangisi küçük?" sorusunu sayar. Tahmin edin: en az ve en çok kaç karşılaştırma? (Örnek 1'in sınırları: min(na, nb) ve na + nb − 1.) Hangi desteler en az karşılaştırmayı verir?
- **Kararlılık.** Aynı değerden iki kart farklı renkteyse (kırmızı 7 solda, siyah 7 sağda), eşitlikte hangisini önce almalısınız ki renk sırası korunsun?
- **Canlı birleştirmeli sıralama (16 kişi).** Her öğrenci bir kart tutar ve sıraya girer. Öğretim üyesi "böl" der: grup ikiye ayrılır, ta ki herkes tek kalana dek. Sonra düzey düzey ikili gruplar birleşir; her birleştirmede bir sayıcı karşılaştırmaları sayar ve tahtaya **düzey toplamını** yazar. Toplam en çok W(16) = 49 olmalıdır (Kavram 1); her düzeyde en çok 16 kişi işe katılır, 4 düzey vardır.
- **Tartışma.** Düzey toplamları neden hep ≤ n'dir? Hızlı sıralamayı aynı grupla canlandırsaydınız (pivot: sıradaki ilk kişi), sıralı dizilmiş bir grupta ne olurdu? Web sayfasındaki birleştirmeli sıralama ağaç oynatıcısıyla sonucu karşılaştırın.

11.9 Atölye 2 · Pivot avı: hızlı sıralamayı karesel davranışa zorlayın (gruplar, ≈ 25 dk)

Kavram 4 (`kodlar/kavram/H06_K04_pivot_avi.c`) Örnek 6'nın Lomuto + küçük parçaya özyineleme sürümünü üç pivot stratejisiyle çalıştırır ve oran = karşılaştırma / (n(n − 1)/2) yazar; 1,00 tam karesel demektir. Argümansız çalıştırınca hazır desenleri, `./H06_K04_pivot_avi girdi.txt` ile **sizin** girdinizi (boşlukla ayrılmış tamsayılar) ölçer. Başlangıç değerleri (n = 1000):

Tablo 6.12
Pivot avı başlangıç tablosu: karşılaştırma (oran), n = 1000 (Kavram 4)

Desen	ilk	rastgele	ortanca3
sıralı	499 500 (1,00)	10 898 (0,02)	9 009 (0,02)
ters	499 500 (1,00)	10 224 (0,02)	15 773 (0,03)
organ borusu	84 744 (0,17)	10 406 (0,02)	85 752 (0,17)
tümü eşit	499 500 (1,00)	499 500 (1,00)	502 497 (1,01)
rastgele	10 757 (0,02)	11 433 (0,02)	10 759 (0,02)

ALGORİTMA ATÖLYESİ · **Görev ve kurallar**

- **Tur 1 (10 dk).** İlk eleman ve rastgele pivot için oranı en yükseğe çıkaran n = 1000 elemanlık girdiler tasarlayın. Rastgele pivotu neden "yenemiyorsunuz"? Tümü eşit girdi üç stratejiyi birden çökertiyor: sorun pivotta mı, bölümlemede mi?
- **Tur 2 (15 dk). Üçün ortancasını** zorlayın: yalnızca birbirinden farklı tamsayılar kullanın (tümü eşit ya da az farklı anahtar yasak). Organ borusunun 0,17 oranını geçin. İpucu: pivot a[lo], a[mid], a[hi]'nin ortancasıdır; bölümlemeden sonra parçaların uç ve orta elemanları yine sizin denetiminizdedir. McIlroy (1999) bu işi girdiyi önceden yazmadan, karşılaştırmalara **yanıt verirken** kurgulayarak yapar: "katil rakip".
- **Değerlendirme.** En yüksek oranı bulan grup girdisini ve fikrini anlatır; sınıf, aynı girdinin rastgele pivotla oranını ölçer. Tartışma: introsort bu saldırıya karşı neyi güvenceye alır, neyi almaz (Bölüm 9)?

11.10 Atölye 3 · Büyük sıralama yarışı: 10^7 tamsayıyı en hızlı kim sıralar? (gruplar, ≈ 30 dk)

Her grup bir **algoritma × dil × eniyileme** yapılandırması seçer ve n = 10^7 rastgele tamsayıyı (ortak LCG, tohum 42) sıralar. Ölçüm Örnek 12'nin ölçüm kipiyle yapılır: `./H06_12_buyuk_deney ALGORITMA rastgele 10000000 3` (ALGORITMA: birlestirmeli, hizli_ilk, hizli_rastgele, hizli_ortanca3, kutuphane); çıktıdaki CSV satırında `islemci_ms` ve `dogru = 1` okunur. Python'da 10^7 çok uzun sürebilir: n = 10^6 ölçüp n lg n ile kestirin ve kestirimi **işaretleyin**. Kurallar: aynı girdi, sonuç doğrulanmış, üç tekrarın ortancası, işlemci süresi; derleyici bayrağı serbesttir (-O0, -O2, -O3, -march=native).

Tablo 6.13
*Yarış tablosu: n = 10^7 rastgele girdi (h06_deney_ozet.csv; * kestirim, alt sınır)*

Yapılandırma	Bu makinede (işlemci süresi)	Sizin makinenizde
C -O2 · hızlı · üçün ortancası	1,11 s	
C -O2 · birleştirmeli	1,51 s	
C -O2 · qsort	1,96 s	
C++ -O2 · std::sort	993,5 ms	
C++ -O2 · hızlı · üçün ortancası	1,08 s	
C -O0 · hızlı · rastgele	n = 10^6: 231,5 ms	
Python · list.sort	≈ 6,8 s*	
Sizin seçiminiz (ör. -O3 -march=native, melez eşik)	?	

Tartışma soruları: (1) Hangi fark algoritmadan, hangisi dilden ya da derleyiciden geliyor? Aynı algoritmanın C ve Python süreleri arasındaki kat ile aynı dildeki iki algoritma arasındaki katı karşılaştırın. (2) `std::sort` neden bizim C++ sürümümüzden hızlı? (3) Grubunuzun ölçümü bu makinedekinden ne kadar farklı ve neden? Ölçüm ortamınızı (işlemci, derleyici, bayraklar) tabloya ekleyin.

12. Kendini Değerlendirme

Aşağıdakilerin her birini kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün.

☐ Bir problemi böl – fethet – birleştir adımlarıyla ifade edip işin hangi adımda yapıldığını söyleyebiliyorum. (3.1)

- ☐ İki sıralı diziyi doğrusal zamanda, kararlı biçimde birleştirebiliyor ve birleştirme değişmezini yazabiliyorum. (4.1, HA1, HA6)
- ☐ Birleştirmeli sıralamanın Θ(n log n) olduğunu özyineleme ağacıyla gösterebiliyor; çağrı (2n − 1) ve derinlik (⌈lg n⌉ + 1) sayılarını hesaplayabiliyorum. (3.3, 4.1)
- ☐ Lomuto ve Hoare bölümlemesini elle izleyebiliyor, Hoare'un dönüş indisinin pivotun yeri olmadığını açıklayabiliyorum. (4.2, HA5)
- ☐ Pivot seçiminin en iyi, ortalama (≈ 1,39 n lg n) ve en kötü (n(n − 1)/2) duruma etkisini ve derinliği O(log n)'de tutmanın yolunu açıklayabiliyorum. (4.2, 3.6, HA8)
- ☐ $T(n) = a \cdot T(n/b) + \Theta(n^d)$ bağıntılarını ana teoremle, kalıba uymayanları yerine koyma ya da özyineleme ağacıyla çözebiliyor; Ω(n log n) alt sınırını karar ağacıyla açıklayabiliyorum. (3.2–3.5)
- ☐ Quickselect ile k'ıncı en küçüğü bulabiliyor, ters çevrimleri birleştirirken sayabiliyorum. (4.3, 4.4)
- ☐ Python'da dilimin kopya olduğunu, C++ `std::sort`'un kararsız olduğunu ve C `qsort`'un algoritmasının belirtilmediğini biliyorum. (8)

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · Bu haftanın konusu nerede derinleşiyor?

- **H7:** Ω(n log n) yalnızca karşılaştırmaya dayalı sıralamalar içindir: küçük tamsayı anahtarları (notlar 0–100) sayma ve taban sıralaması doğrusal zamanda sıralar; öbek sıralaması introsort'un "yedek" algoritmasıdır. **H13:** alt problemler örtüşürse dinamik programlama tekrarı tabloyla kaldırır.
- Ana teorem ve yineleme bağıntıları, Algoritma Analizi dersinde Akra–Bazzi gibi genellemelerle sürer.
- Böl ve fethet; hızlı Fourier dönüşümü, Strassen matris çarpımı ve en yakın nokta çifti algoritmalarının temelidir.
- Dış bellek sıralaması: belleğe sığmayan verinin birleştirmeli sıralamayla sıralanması (veritabanı dersleri).

MERAK KUTUSU · 1,39 nereden geliyor? Medyanların medyanı nedir?

Ortalama karşılaştırma sayısı. Lomuto bölümlemesi n − 1 karşılaştırma yapar ve rastgele girdide pivotun sırası 0 … n − 1 arasında eşit olasılıklıdır: $C(n) = (n-1) + (1/n)\cdot\Sigma\,[C(k) + C(n-1-k)] = (n-1) + (2/n)\cdot\Sigma\,C(k)$. n ile çarpıp n − 1 için yazılan eşitlikten çıkarınca $nC(n) = (n+1)C(n-1) + 2(n-1)$; n(n + 1)'e bölünce toplam teleskoplaşır ve $C(n) = 2(n+1)H(n) - 4n \approx 2n \ln n = (2 \ln 2)\cdot n \lg n \approx 1{,}386\cdot n \lg n$ çıkar (Knuth, 1998; Sedgewick & Wayne, 2011).

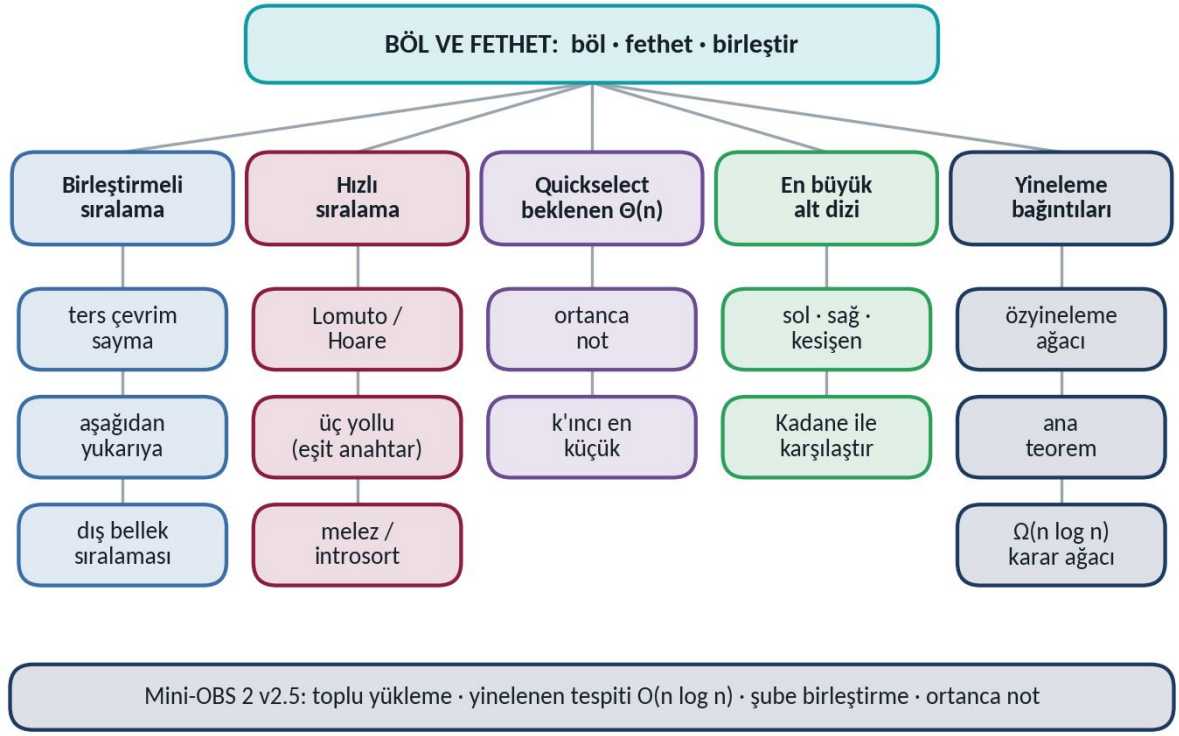
Medyanların medyanı. Quickselect'in en kötü durumu da doğrusal yapılabilir: elemanları beşerli gruplara ayır, her grubun ortancasını bul, bu ⌈n/5⌉ ortancanın ortancasını **özyinelemeyle** seç ve pivot yap. Bu pivotun her iki yanında en az ≈ 3n/10 eleman bulunur; bağıntı $T(n) \le T(n/5) + T(7n/10) + O(n)$'dir ve 1/5 + 7/10 < 1 olduğu için özyineleme ağacının düzey işleri geometrik azalır: T(n) = Θ(n) (Blum vd., 1973). Sabit çarpanı büyük olduğundan pratikte rastgele pivot tercih edilir.

14. Özet, Kavram Haritası ve Terim Sözlüğü

- Böl ve fethet: böl, fethet (özyineleme), birleştir. Maliyet, işin bu üç adıma dağılımından okunur: birleştirmeli sıralamada iş birleştirmede, hızlı sıralamada bölümlemededir.
- Yineleme bağıntıları yerine koyma ve özyineleme ağacıyla çözülür; $T(n) = aT(n/b) + \Theta(n^d)$ kalıbında ana teorem a ile b^d'yi karşılaştırır: kök baskın $\Theta(n^d)$, düzeyler eşit $\Theta(n^d \log n)$, yapraklar baskın Θ(n^(log_b a)).

- Birleştirmeli sıralama her girdide Θ(n log n), kararlı, O(n) ek bellek; tampon bir kez ayrılır. Karşılaştırma sayısı ≤ n⌈lg n⌉ − 2^⌈lg n⌉ + 1, karar ağacının alt sınırı ⌈lg n!⌉'e (Ω(n log n)) çok yakın.
- Hızlı sıralama yerinde ve ortalamada ≈ 1,39 n lg n karşılaştırma yapar; pivot hep uçta olursa Θ(n^2). Rastgele pivot ya da üçün ortancası, küçük parçaya özyineleme (derinlik ≤ ⌊lg n⌋ + 1) ve eşit anahtarlar için üç yollu bölümleme onu güvenilir kılar.
- Aynı fikir ters çevrim sayısını (birleştirirken say), k'ıncı en küçüğü (quickselect, ortalama Θ(n)) ve en büyük alt diziyi (üç aday) de çözer. Kütüphaneler melezdir: introsort (C++), Timsort/Powersort (Python).

Şekil 6.15
Hafta 6 kavram haritası



Tablo 6.14
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
böl ve fethet; böl, fethet, birleştir	divide and conquer; divide, conquer, combine	
birleştirme; birleştirmeli sıralama	merge; merge sort	yukarıdan aşağıya / aşağıdan yukarıya
bölümleme; pivot; hızlı sıralama	partition; pivot; quicksort	Lomuto, Hoare
üçün ortancası; üç yollu bölümleme	median-of-three; three-way partitioning	Hollanda bayrağı
yineleme bağıntısı; yerine koyma yöntemi	recurrence; substitution method	
özyineleme ağacı; ana teorem	recursion tree; master theorem	basitleştirilmiş biçim
karar ağacı; alt sınır	decision tree; lower bound	Ω(n log n)
kararlı; yerinde; uyarlanabilir	stable; in-place; adaptive	

Türkçe	İngilizce	Not
ters çevrim	inversion	araya ekleme: kaydırma sayısı
seçim problemi; ortanca; melez sıralama; eşik	selection problem; median; hybrid sort; cutoff	quickselect; introsort
kuyruk çağrısı eleme; yığın taşması	tail call elimination; stack overflow	derleyici eniyilemesi
takas; ödünleşim	swap; trade-off	"takas" yalnızca swap
kâhin; katil rakip	test oracle; killer adversary	McIlroy (1999)

15. Kaynaklar

Bentley, J. L., & McIlroy, M. D. (1993). Engineering a sort function. *Software: Practice and Experience*, *23*(11), 1249–1265. https://doi.org/10.1002/spe.4380231105

Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., & Tarjan, R. E. (1973). Time bounds for selection. *Journal of Computer and System Sciences*, *7*(4), 448–461. https://doi.org/10.1016/S0022-0000(73)80033-9

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Crosby, S. A., & Wallach, D. S. (2003). Denial of service via algorithmic complexity attacks. In *Proceedings of the 12th USENIX Security Symposium* (pp. 29–44). USENIX Association.

Dijkstra, E. W. (1976). *A discipline of programming*. Prentice-Hall.

Hoare, C. A. R. (1961a). Algorithm 64: Quicksort. *Communications of the ACM*, *4*(7), 321.

Hoare, C. A. R. (1961b). Algorithm 65: Find. *Communications of the ACM*, *4*(7), 321–322.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, *5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Hoare, C. A. R. (1981). The emperor's old clothes. *Communications of the ACM*, *24*(2), 75–83. https://doi.org/10.1145/358549.358561

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson/Addison-Wesley.

Knuth, D. E. (1970). Von Neumann's first computer program. *Computing Surveys*, *2*(4), 247–260.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

McIlroy, M. D. (1999). A killer adversary for quicksort. *Software: Practice and Experience*, *29*(4), 341–344.

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, *27*(8), 983–993.

Sedgewick, R. (1998). *Algorithms in C, parts 1–4: Fundamentals, data structures, sorting, searching* (3rd ed.). Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Shustek, L. (2009). Interview: An interview with C.A.R. Hoare. *Communications of the ACM*, *52*(3), 38–41. https://doi.org/10.1145/1467247.1467261

Spicer, D. (2026, March 11). *In memoriam: Sir Antony Hoare (1934–2026)*. Computer History Museum. https://computerhistory.org/blog/in-memoriam-sir-antony-hoare-1934-2026/

University of Liverpool. (2022, December 12). *Liverpool computer scientists improve Python sorting function*. https://news.liverpool.ac.uk/2022/12/12/liverpool-computer-scientists-improve-python-sorting-function/

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden algoritmanın çekirdeğini oluşturan fonksiyonlar, dosyadaki satır numaralarıyla verilir (Bölüm 4'ün "Üç dil karşılaştırması" adımlarında gösterilen kesitler burada yinelenmez, aynı dosyanın öteki çekirdek fonksiyonu verilir); tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır; rastgele girdiler ve rastgele pivotlar üç dilde de ortak LCG'den (`ortak_rastgele`) gelir. On üç örneğin hepsinin çıktısı üç dilde bayt bayt aynıdır (=); `kodlar/esdeger_test.py` bunu denetler.

Tablo 6.15
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H06_02_birlestirmeli_siralama)

Dil	Derleme / çalıştırma	Ek dosya ya da bayrak
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c ortak_rastgele.c -o X -lm · ./X`	Örnek 11, 12: + zamanlayici.c
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Hepsi	`Örnek 11, 12 ölçüm kipi: ./X N TEKRAR · ./X ALGORITMA GIRDI N TEKRAR`	süreler stderr'e; stdout üç dilde aynı
Deney	`cd deney && python3 olcum_calistir.py && python3 kestirim.py`	sonuclar/*.csv yeniden üretilir

A.1 Örnek 1 · İki sıralı dizinin birleştirilmesi: birleştirme değişmezi ve eşitlikte soldan alarak kararlılık

C++ cpp/H06_01_birlestirme.cpp · satır 28–47

```cpp
void birlestir(const std::vector<int>& a, const std::vector<int>& b,
               std::vector<int>& c)
{
    std::size_t na = a.size(), nb = b.size();
    std::size_t i = 0, j = 0, k = 0;
    while (i < na && j < nb) {
        karsilastirma++;
        if (a[i] <= b[j]) {  // eşitse soldan
            c[k++] = a[i++];
        } else {
            c[k++] = b[j++];
        }
    }
    while (i < na) {  // sol dizinin kalanı
        c[k++] = a[i++];
    }
    while (j < nb) {  // sağ dizinin kalanı
        c[k++] = b[j++];
    }
}
```

Python `python/H06_01_birlestirme.py · satır 16–35`

```python
def birlestir(a, b, c):
    global karsilastirma
    i, j, k, na, nb = 0, 0, 0, len(a), len(b)
    while i < na and j < nb:
        karsilastirma += 1
        if a[i] <= b[j]:  # eşitse soldan
            c[k] = a[i]
            i += 1
        else:
            c[k] = b[j]
            j += 1
        k += 1
    while i < na:  # sol dizinin kalanı
        c[k] = a[i]
        i += 1
        k += 1
    while j < nb:  # sağ dizinin kalanı
        c[k] = b[j]
        j += 1
        k += 1
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de dizinin uzunluğu ayrı parametredir (`na`, `nb`); C++'ta `a.size()`, Python'da `len(a)` diziden okunur. C ve C++'taki `c[k++] = a[i++]` Python'da üç ayrı satırdır (`++` yoktur). Kâhinde birleşik dizi C'de iki döngüyle, C++'ta `insert`, Python'da `x + y` ile kurulur; vektör/liste eşitliği `==` ile tek adımda denetlenir. (Fonksiyon dışındaki fark satırları: C++ 165, 166, 167; Python 76, 138, 139.)

A.2 Örnek 2 · Birleştirmeli sıralama (yukarıdan aşağıya, tek yardımcı tampon): karşılaştırma, kopya, çağrı ve derinlik

C++ `cpp/H06_02_birlestirmeli_siralama.cpp · satır 51–66`

```cpp
static void bs_ozyineli(std::vector<int>& a, std::vector<int>& yrd, int lo,
                        int hi, int d)
{
    cagri++;
    if (d > en_derin) {
        en_derin = d;
    }
    if (lo >= hi) {  // 0 ya da 1 eleman: sıralı
        return;
    }
    int mid = lo + (hi - lo) / 2;
    // böl ve fethet: iki yarıyı ayrı ayrı sırala
    bs_ozyineli(a, yrd, lo, mid, d + 1);
    bs_ozyineli(a, yrd, mid + 1, hi, d + 1);
    birlestir(a, yrd, lo, mid, hi);  // birleştir
}
```

Python **python/H06_02_birlestirmeli_siralama.py · satır 44–55**

```python
def bs_ozyineli(a, yrd, lo, hi, d):
    global cagri, en_derin
    cagri += 1
    if d > en_derin:
        en_derin = d
    if lo >= hi:  # 0 ya da 1 eleman: sıralı
        return
    mid = lo + (hi - lo) // 2
    # böl ve fethet: iki yarıyı ayrı ayrı sırala
    bs_ozyineli(a, yrd, lo, mid, d + 1)
    bs_ozyineli(a, yrd, mid + 1, hi, d + 1)
    birlestir(a, yrd, lo, mid, hi)  # birleştir
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Tek tampon C'de `malloc` ile bir kez ayrılıp `free` ile bırakılır; C++'ta `std::vector<int> yrd(a.size())` kapsam bitince kendiliğinden bırakılır (RAII); Python'da liste çöp toplayıcıya kalır. Python'da orta nokta `//` ile hesaplanır ve sayaçları güncellemek için `global` gerekir. (Fonksiyon dışındaki fark satırları: C++ 69, 74, 109, 112; Python 19, 38, 63, 88.)

A.3 Örnek 3 · Özyineleme ağacı izleme: düzey başına iş sayacı ve ana teoremin üç durumu

C++ **cpp/H06_03_ozyineleme_agaci.cpp · satır 160–172**

```cpp
static void yineleme(long long n, int a, int b, int d, int duzey)
{
    if (duzey > duzey_sayisi) {
        duzey_sayisi = duzey;
    }
    duzey_is[duzey] += us(n, d);
    if (n <= 1) {
        return;
    }
    for (int k = 0; k < a; k++) {
        yineleme(n / b, a, b, d, duzey + 1);
    }
}
```

Python **python/H06_03_ozyineleme_agaci.py · satır 135–143**

```python
def yineleme(n, a, b, d, duzey):
    global duzey_sayisi
    if duzey > duzey_sayisi:
        duzey_sayisi = duzey
    duzey_is[duzey] += us(n, d)
    if n <= 1:
        return
    for _ in range(a):
        yineleme(n // b, a, b, d, duzey + 1)
```

DİL PENCERESİ · **Farklılaşan satırlar**

Düzey dizileri C'de sabit boyutlu `static` dizi, C++'ta `std::vector`, Python'da liste; Python'da bu listelerin öğelerini değiştirmek `global` istemez, ama `duzey_sayisi` adını yeniden bağlamak ister. Boyut aralığı metni C'de `snprintf`, C++'ta `std::to_string`, Python'da f-string ile kurulur; `n / b` tamsayı bölmesi Python'da `//`'dir. (Fonksiyon dışındaki fark satırları: C++ 21, 35, 135, 191; Python 30, 61, 118, 158.)

A.4 Örnek 4 · Lomuto bölümleme (pivot = a[hi]) ve hızlı sıralama: değişmez, izleme ve 2(n+1)H(n) - 4n beklentisi

C++ **cpp/H06_04_lomuto_hizli.cpp · satır 45–56**

```cpp
45 void hizli_sirala(std::vector<int>& a, int lo, int hi, int d)
46 {
47     if (d > en_derin) {
48         en_derin = d;
49     }
50     if (lo >= hi) {
51         return;
52     }
53     int p = lomuto_bolumle(a, lo, hi);
54     hizli_sirala(a, lo, p - 1, d + 1);
55     hizli_sirala(a, p + 1, hi, d + 1);
56 }
```

Python **python/H06_04_lomuto_hizli.py · satır 37–45**

```python
37 def hizli_sirala(a, lo, hi, d):
38     global en_derin
39     if d > en_derin:
40         en_derin = d
41     if lo >= hi:
42         return
43     p = lomuto_bolumle(a, lo, hi)
44     hizli_sirala(a, lo, p - 1, d + 1)
45     hizli_sirala(a, p + 1, hi, d + 1)
```

DİL PENCERESİ · **Farklılaşan satırlar**

Takas C'de geçici değişkenle, C++'ta `std::swap`, Python'da çoklu atamayla (`a[i], a[j] = a[j], a[i]`) yapılır; üçünde de sayaç aynı yerde artar. Python'da `range(lo, hi)` üst sınırı dışarıda bırakır: C'deki `j < hi` koşulunun karşılığıdır. (Fonksiyon dışındaki fark satırları: C++ 24, 29, 140; Python 18, 27, 75, 122.)

A.5 Örnek 5 · Hoare bölümleme ve Lomuto ile karşılaştırma/takas sayısı; dönüş indisi pivotun yeri değildir

C++ cpp/H06_05_hoare_bolumleme.cpp · satır 28–46

```cpp
int hoare_bolumle(std::vector<int>& a, int lo, int hi)
{
    int x = a[lo];
    int i = lo - 1, j = hi + 1;
    for (;;) {
        do {  // sağdan: x'ten büyük olmayanı bul
            j--;
            karsilastirma++;
        } while (a[j] > x);
        do {  // soldan: x'ten küçük olmayanı bul
            i++;
            karsilastirma++;
        } while (a[i] < x);
        if (i >= j) {
            return j;
        }
        takas(a, i, j);
    }
}
```

Python python/H06_05_hoare_bolumleme.py · satır 23–40

```python
def hoare_bolumle(a, lo, hi):
    global karsilastirma
    x = a[lo]
    i, j = lo - 1, hi + 1
    while True:
        j -= 1  # sağdan: x'ten büyük olmayanı bul
        karsilastirma += 1
        while a[j] > x:  # (Python'da do-while yok)
            j -= 1
            karsilastirma += 1
        i += 1  # soldan: x'ten küçük olmayanı bul
        karsilastirma += 1
        while a[i] < x:
            i += 1
            karsilastirma += 1
        if i >= j:
            return j
        takas(a, i, j)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'taki `do { j--; } while (a[j] > x);` döngüsü Python'da yoktur: önce bir adım atılır, ardından `while` koşulu sınanır (sayaçlar aynı kalır). Kopya C'de `malloc` + döngü, C++'ta `std::vector<int> b = a`, Python'da `a[:]` ile alınır. (Fonksiyon dışındaki fark satırları: C++ 23, 136, 190; Python 116.)

A.6 Örnek 6 · Pivot deneyi: ilk eleman / rastgele / üçün ortancası × sıralı, ters, rastgele girdi; ağaç yüksekliği ve yığın derinliği

C++ cpp/H06_06_pivot_deneyi.cpp · satır 51–60

```cpp
static int pivot_sec(const std::vector<int>& a, int lo, int hi, int strateji)
{
    if (strateji == ILK) {
        return lo;
    }
    if (strateji == RASTGELE) {
        return rastgele_aralik(lo, hi);
    }
    return ortanca3(a, lo, lo + (hi - lo) / 2, hi);
}
```

Python python/H06_06_pivot_deneyi.py · satır 43–48

```python
def pivot_sec(a, lo, hi, strateji):
    if strateji == ILK:
        return lo
    if strateji == RASTGELE:
        return rastgele_aralik(lo, hi)
    return ortanca3(a, lo, lo + (hi - lo) // 2, hi)
```

> DİL PENCERESİ • **Farklılaşan satırlar**
>
> Rastgele pivot üç dilde ortak LCG'den (`rastgele_aralik`) gelir; C'nin `rand()`'ı, C++'ın `std::mt19937`'si ve Python'un `random`'ı aynı diziyi vermezdi. `naif N` kipinde Python derinlik ~1000'de `RecursionError` fırlatır ve yakalanır; C/C++'ta sınır yığın boyutudur. Çökme öncesi satırın görünmesi için C'de `fflush`, C++'ta `std::endl`, Python'da `flush=True` kullanılır. (Fonksiyon dışındaki fark satırları: C++ 65, 153, 167, 224; Python 102, 129, 132, 197.)

A.7 Örnek 7 · Üç yollu bölümleme (Hollanda bayrağı, Dijkstra) ve çok tekrar eden anahtarlarda iki yollu ile karşılaştırma

C++ cpp/H06_07_uc_yollu_bolumleme.cpp · satır 30–51

```cpp
std::pair<int, int> uc_yollu_bolumle(std::vector<int>& a, int lo, int hi,
                                     int p)
{
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        karsilastirma++;
        if (a[i] < p) {  // küçük: sola
            takas(a, lt, i);
            lt++;
            i++;
        } else {
            karsilastirma++;
            if (a[i] > p) {  // büyük: sağa
                takas(a, i, gt);
                gt--;
```

```
            } else {  // eşit: yerinde kalır
                i++;
            }
        }
    }
    return {lt, gt};
}
```

Python `python/H06_07_uc_yollu_bolumleme.py · satır 24–40`

```python
def uc_yollu_bolumle(a, lo, hi, p):
    global karsilastirma
    lt, i, gt = lo, lo, hi
    while i <= gt:
        karsilastirma += 1
        if a[i] < p:  # küçük: sola
            takas(a, lt, i)
            lt += 1
            i += 1
        else:
            karsilastirma += 1
            if a[i] > p:  # büyük: sağa
                takas(a, i, gt)
                gt -= 1
            else:  # eşit: yerinde kalır
                i += 1
    return lt, gt
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Bölümleme iki sınır (lt, gt) döndürür: C'de çıkış parametreleri (`int *lt_c, int *gt_c`), C++17'de `std::pair` ve yapılandırılmış bağlama (`auto [lt, gt] = ...`), Python'da demet (`return lt, gt`) kullanılır. (Fonksiyon dışındaki fark satırları: C++ 63; Python 51.)

A.8 Örnek 8 · Ters çevrim sayısı: birleştirmeli sıralamayla Θ(n log n); H5'in Θ(n²) çift taraması kâhin

C++ `cpp/H06_08_ters_cevrim.cpp · satır 21–45`

```cpp
static long long birlestir_say(std::vector<int>& a, std::vector<int>& yrd,
                               int lo, int mid, int hi)
{
    long long sayi = 0;
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) {
        karsilastirma++;
        if (a[i] <= a[j]) {  // eşitse sayılmaz
            yrd[k++] = a[i++];
        } else {
            yrd[k++] = a[j++];
            sayi += mid - i + 1;  // hepsi > a[j]
        }
    }
```

```
    while (i <= mid) {
        yrd[k++] = a[i++];
    }
    while (j <= hi) {
        yrd[k++] = a[j++];
    }
    for (k = lo; k <= hi; k++) {
        a[k] = yrd[k];
    }
    return sayi;
}
```

Python `python/H06_08_ters_cevrim.py · satır 17–41`

```python
def birlestir_say(a, yrd, lo, mid, hi):
    global karsilastirma
    sayi = 0
    i, j, k = lo, mid + 1, lo
    while i <= mid and j <= hi:
        karsilastirma += 1
        if a[i] <= a[j]:  # eşitse sayılmaz
            yrd[k] = a[i]
            i += 1
        else:
            yrd[k] = a[j]
            j += 1
            sayi += mid - i + 1  # hepsi > a[j]
        k += 1
    while i <= mid:
        yrd[k] = a[i]
        i += 1
        k += 1
    while j <= hi:
        yrd[k] = a[j]
        j += 1
        k += 1
    for k in range(lo, hi + 1):
        a[k] = yrd[k]
    return sayi
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ters çevrim sayısı n = 100 000'de $5 \cdot 10^9$'a yaklaşır: C ve C++'ta `long long` gerekir (`int` taşar), Python tamsayıları sınırsızdır. Girdiyi bozmamak için kopya C'de `malloc` + döngüyle, C++'ta `std::vector<int> b = a`, Python'da `a[:]` ile alınır. (Fonksiyon dışındaki fark satırları: C++ 61, 63; Python 48, 56, 57, 115.)

A.9 Örnek 9 · Quickselect: ortanca not (üç yollu bölümleme, rastgele pivot, yinelemeli); sıralama kâhine karşı

C++ cpp/H06_09_quickselect.cpp · satır 25–43

```cpp
static std::pair<int, int> uc_yollu_bolumle(std::vector<int>& a, int lo,
                                            int hi, int p)
{
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        karsilastirma++;
        if (a[i] < p) {
            std::swap(a[lt++], a[i++]);
        } else {
            karsilastirma++;
            if (a[i] > p) {
                std::swap(a[i], a[gt--]);
            } else {
                i++;
            }
        }
    }
    return {lt, gt};
}
```

Python python/H06_09_quickselect.py · satır 20–36

```python
def uc_yollu_bolumle(a, lo, hi, p):
    global karsilastirma
    lt, i, gt = lo, lo, hi
    while i <= gt:
        karsilastirma += 1
        if a[i] < p:
            a[lt], a[i] = a[i], a[lt]
            lt += 1
            i += 1
        else:
            karsilastirma += 1
            if a[i] > p:
                a[i], a[gt] = a[gt], a[i]
                gt -= 1
            else:
                i += 1
    return lt, gt
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta `takas(a, lt++, i++)` iki indisi aynı satırda ilerletir; Python'da `++` olmadığından takas ve artırmalar ayrı satırlardır. Kütüphane karşılığı C++'ta `std::nth_element`'tir; Python'da `statistics.median` sıralayarak çalışır (Θ(n log n)). (Fonksiyon dışındaki fark satırları: C++ 52, 123; Python 107.)

A.10 Örnek 10 · Aşağıdan yukarıya (yinelemeli) birleştirmeli sıralama: geçişler, eşsiz son blok ve yukarıdan aşağıya ile karşılaştırma

C++ cpp/H06_10_asagidan_yukari.cpp · satır 55–82

```cpp
int asagidan_yukari_sirala(std::vector<int>& a, bool yaz)
{
    int n = static_cast<int>(a.size());
    std::vector<int> yrd(a.size());
    int gecis = 0;
    for (int gen = 1; gen < n; gen *= 2) {
        int onceki = birlestirme_sayisi;
        for (int lo = 0; lo < n - gen; lo += 2 * gen) {
            int mid = lo + gen - 1;
            int hi = std::min(lo + 2 * gen - 1, n - 1);
            birlestir(a, yrd, lo, mid, hi);
        }
        gecis++;
        if (yaz) {
            int blok = (n + gen - 1) / gen;
            std::cout << "gen = " << std::setw(2) << gen << " :";
            dizi_yaz(a);
            std::cout << "   (" << birlestirme_sayisi - onceki
                      << " birleştirme";
            if (blok % 2 == 1) {
                std::cout << "; [" << (blok - 1) * gen << ".." << n - 1
                          << "] eşsiz";
            }
            std::cout << ")\n";
        }
    }
    return gecis;
}
```

Python python/H06_10_asagidan_yukari.py · satır 48–69

```python
def asagidan_yukari_sirala(a, yaz):
    n = len(a)
    yrd = [0] * n
    gecis = 0
    gen = 1
    while gen < n:
        onceki = birlestirme_sayisi
        for lo in range(0, n - gen, 2 * gen):
            mid = lo + gen - 1
            hi = min(lo + 2 * gen - 1, n - 1)
            birlestir(a, yrd, lo, mid, hi)
        gecis += 1
        if yaz:
            blok = (n + gen - 1) // gen
            print(f"gen = {gen:2d} :", end="")
            dizi_yaz(a)
            print(f"   ({birlestirme_sayisi - onceki} birleştirme", end="")
            if blok % 2 == 1:
                print(f"; [{(blok - 1) * gen}..{n - 1}] eşsiz", end="")
            print(")")
```

```
68          gen *= 2
69      return gecis
```

DİL PENCERESİ · Farklılaşan satırlar

C'deki `for (gen = 1; gen < n; gen *= 2)` Python'da `range` ile yazılamaz (adım çarpımsal): `while` döngüsü kullanılır; iç döngü ise `range(0, n - gen, 2 * gen)` olur. Bloğun sonu C'de koşullu ifadeyle, C++'ta `std::min`, Python'da `min` ile bulunur. (Fonksiyon dışındaki fark satırları: C++ 138.)

A.11 Örnek 11 · Melez sıralama: hızlı sıralama + küçük parçalarda araya ekleme; eşik deneyi (4...64; sayaç ve süre)

C++ **cpp/H06_11_melez_siralama.cpp · satır 86–101**

```cpp
86  void melez_sirala(std::vector<int>& a, int lo, int hi, int esik)
87  {
88      while (hi - lo + 1 > esik) {
89          int p = bolumle(a, lo, hi);
90          if (p - lo < hi - p) {
91              melez_sirala(a, lo, p - 1, esik);
92              lo = p + 1;
93          } else {
94              melez_sirala(a, p + 1, hi, esik);
95              hi = p - 1;
96          }
97      }
98      if (lo < hi) {
99          araya_ekle(a, lo, hi);  // kısa parça
100     }
101 }
```

Python **python/H06_11_melez_siralama.py · satır 71–81**

```python
71  def melez_sirala(a, lo, hi, esik):
72      while hi - lo + 1 > esik:
73          p = bolumle(a, lo, hi)
74          if p - lo < hi - p:
75              melez_sirala(a, lo, p - 1, esik)
76              lo = p + 1
77          else:
78              melez_sirala(a, p + 1, hi, esik)
79              hi = p - 1
80      if lo < hi:
81          araya_ekle(a, lo, hi)  # kısa parça
```

DİL PENCERESİ · Farklılaşan satırlar

Ölçülecek iş C'de `void (*)(void *)` işaretçisi + yapı (`Gorev`), C++'ta yakalayan lambda (`[&] { ... }`), Python'da iç fonksiyonla `olc_ortanca`'ya verilir. Kopyalama C'de `memcpy`, C++'ta vektör ataması, Python'da `calisma[:] = orijinal` (aynı listeye yerinde kopya) ile yapılır. Süreler stderr'e yazıldığı için stdout aynıdır. (Fonksiyon dışındaki fark satırları: C++ 159, 160, 168; Python 123, 124, 131.)

A.12 Örnek 12 · Büyük deney programı: birleştirmeli / hızlı (3 pivot) / kütüphane / araya ekleme; varsayılan küçük, ölçüm kipi CSV

C++ cpp/H06_12_buyuk_deney.cpp · satır 152–167

```cpp
static void sirala(std::vector<int>& a, std::vector<int>& yrd, int alg)
{
    int n = static_cast<int>(a.size());
    karsilastirma = 0;
    agac_yukseklig = yigin_derinligi = 0;
    rastgele_tohum(2026);               // rastgele pivot dizisi sabit
    if (alg == BIRLESTIRMELI) {
        bs_ozyineli(a, yrd, 0, n - 1, 1);
    } else if (alg == KUTUPHANE) {
        std::sort(a.begin(), a.end());
    } else if (alg == ARAYA_EKLEME) {
        araya_ekleme(a);
    } else {
        hizli_sirala(a, 0, n - 1, alg, 1, 1);
    }
}
```

Python python/H06_12_buyuk_deney.py · satır 132–145

```python
def sirala(a, yrd, alg):
    global karsilastirma, agac_yukseklig, yigin_derinligi
    n = len(a)
    karsilastirma = 0
    agac_yukseklig = yigin_derinligi = 0
    rastgele_tohum(2026)                # rastgele pivot dizisi sabit
    if alg == BIRLESTIRMELI:
        bs_ozyineli(a, yrd, 0, n - 1, 1)
    elif alg == KUTUPHANE:
        a.sort()
    elif alg == ARAYA_EKLEME:
        araya_ekleme(a)
    else:
        hizli_sirala(a, 0, n - 1, alg, 1, 1)
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Kütüphane sıralaması C'de karşılaştırıcı işaretçili `qsort`, C++'ta şablonla satır içine alınan `std::sort`, Python'da C ile yazılmış `list.sort` (Timsort)'tur; ölçümde fark buradan doğar. Araya eklemedeki `while (j >= 0 && (karsilastirma++, a[j] > anahtar))` virgül işlecini Python'da `while` + `break` karşılar. (Fonksiyon dışındaki fark satırları: C++ 223, 234; Python 122, 160, 171.)

A.13 Örnek 13 · Böl ve fethet ile en büyük alt dizi toplamı; H2 Kadane ile aynı sonuç, toplama sayısı karşılaştırması

C++ cpp/H06_13_en_buyuk_alt_dizi.cpp · satır 29–54

```cpp
static Aralik kesisen(const std::vector<int>& a, int lo, int mid, int hi)
{
    long long toplam = a[mid], sol_en = a[mid];
    int bas = mid;
    for (int i = mid - 1; i >= lo; i--) {  // sola
        toplam += a[i];
        toplama++;
        if (toplam > sol_en) {
            sol_en = toplam;
            bas = i;
        }
    }
    toplam = a[mid + 1];
    long long sag_en = a[mid + 1];
    int son = mid + 1;
    for (int j = mid + 2; j <= hi; j++) {  // sağa
        toplam += a[j];
        toplama++;
        if (toplam > sag_en) {
            sag_en = toplam;
            son = j;
        }
    }
    toplama++;  // sol_en + sag_en
    return {sol_en + sag_en, bas, son};
}
```

Python python/H06_13_en_buyuk_alt_dizi.py · satır 18–38

```python
def kesisen(a, lo, mid, hi):
    global toplama
    toplam = sol_en = a[mid]
    bas = mid
    for i in range(mid - 1, lo - 1, -1):  # sola
        toplam += a[i]
        toplama += 1
        if toplam > sol_en:
            sol_en = toplam
            bas = i
    toplam = a[mid + 1]
    sag_en = a[mid + 1]
    son = mid + 1
    for j in range(mid + 2, hi + 1):  # sağa
        toplam += a[j]
        toplama += 1
        if toplam > sag_en:
            sag_en = toplam
            son = j
    toplama += 1  # sol_en + sag_en
    return (sol_en + sag_en, bas, son)
```

DİL PENCERESİ · **Farklılaşan satırlar**

Sonuç (toplam, bas, son) C'de `Aralik` yapısı, C++'ta küme başlatmalı (`return {…}`) yapı, Python'da demettir; demet öğelerine `sol[0]` ile erişilir. Geriye doğru döngü Python'da `range(mid - 1, lo - 1, -1)` olur. (Fonksiyon dışındaki fark satırları: C++ 63; Python 53.)